```
// ==================== lib/main.dart ====================
/// 应用入口。
///
/// 启动顺序（有讲究，别改）：
/// 1. 打开数据库 —— 所有页面都依赖它
/// 2. 初始化通知（不请求权限）—— 只准备通道
/// 3. 建默认本地用户
/// 4. 渲染 UI
///
/// 通知权限**不在这里请求**。等用户添加了第一只宠物、看到生成的计划之后再要，
/// 授权率会高得多。见 ui/sheets.dart 的 _AddPetSheetState._submit。
library;

import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:package_info_plus/package_info_plus.dart';

import 'core/l10n.dart';
import 'core/region.dart';
import 'core/theme.dart';
import 'data/db/app_database.dart';
import 'providers.dart';
import 'services/crash_log.dart';
import 'services/notification_service.dart';
import 'ui/me_screen.dart';
import 'ui/profile_screen.dart';
import 'ui/records_screen.dart';
import 'ui/reminder_sheet.dart';
import 'ui/today_screen.dart';
import 'ui/update_flow.dart';

/// 全局 Navigator key。目前只有一处用途：**通知点击后要从 App 外部
/// （原生回调）拿到一个能弹层的 context**。别拿它到处做导航。
final GlobalKey<NavigatorState> appNavigatorKey = GlobalKey<NavigatorState>();

/// 首帧之前来的通知点击先暂存，等 UI 起来再弹。
String? _pendingReminderId;

/// 入口。
///
///  **整个流程包在 `runZonedGuarded` 里，不是只包 runApp。**
///
/// 未 await 的 Future 里抛出的异常，`FlutterError.onError` 与
/// `PlatformDispatcher.onError` **都抓不到** —— 只有 zone 能接。
/// 而「用户点了没反应」这类问题恰好几乎全是这一类：
/// 按钮 `onPressed` 里 `await` 了一个会抛的操作、没人 catch，
/// 于是错误消失得干干净净，日志里什么都没有。
///
```

```
/// 我在排查「documents 没反应」时踩过这个：两个 onError 都装上了，
/// 异常仍然静默，因为 `openFile()` 抛在 await 链里、没被 try 覆盖。
/// 所以这条边界必须在这里守住，**不是**在每个按钮上补 try。
Future<void> main() async {
  runZonedGuarded(
    () async {
      // Binding initialization and runApp must share the same zone so Flutter
      // callbacks retain the startup error handler.
      WidgetsFlutterBinding.ensureInitialized();
      CrashLog.instance.installGlobalHandlers();
      await CrashLog.instance.init();
      unawaited(_stampVersion());
      await _bootstrap();
    },
    (error, stack) {
      debugPrint('[zone] 未捕获异常：$error\n$stack');
      unawaited(
        CrashLog.instance.record(
          CrashEntry(
            at: DateTime.now(),
            kind: 'zone',
            message: error.toString(),
            stack: stack.toString(),
            context: const {},
          ),
        ),
      );
    },
  );
}

/// 读取真实版本号补进崩溃日志。
///
/// 失败就算了 —— 版本信息缺失不该影响启动，且包内一定能读到，
/// 读不到说明环境异常，记一条日志就够了。
Future<void> _stampVersion() async {
  try {
    final info = await PackageInfo.fromPlatform();
    CrashLog.instance.setAppInfo(
      version: '${info.version}+${info.buildNumber}',
      region: AppRegion.current.name,
    );
  } catch (e) {
    debugPrint('[crash_log] 读版本失败：$e');
  }
}

/// 首帧之后再上传崩溃记录。
///
/// ## 为什么延后到首帧之后
```

```
///
/// 上传要走网络，慢的话会拖慢启动。而启动速度是用户直接感知的，
/// 崩溃上报是后台义务 —— 两者冲突时先保证前者。
///
/// ## 为什么要等一会再开始
///
/// 刚启动时往往网络还没就绪（要连 WiFi、握手 TLS、解析 DNS）。
/// 立刻发大概率失败，失败就得等下次启动 —— 白白浪费一次机会。
/// 延迟几秒给网络留出时间。
Future<void> _uploadCrashesLater() async {
  await Future<void>.delayed(const Duration(seconds: 3));
  if (CrashLog.instance.recent.isNotEmpty) {
    await CrashLog.instance.autoUpload();
  }
}

/// 启动流程。与 [main] 分开是因为要整段被 zone 包住。
Future<void> _bootstrap() async {
  // 数据库先就位，避免首帧闪一下空态。
  await AppDatabase.instance.open();
  await NotificationService.instance.init();
  await _ensureLocalUser();

  // 点通知进 App → 直接弹这条提醒的操作卡（完成 / 明天再说）。
  //
  // onReminderTapped 是 NotificationService 上的**静态**字段，必须走类名赋值。
  // 写成 NotificationService.instance.onReminderTapped = ... 会被 analyzer 判
  // `instance_access_to_static_member`（在 Dart 里是 error，构建直接挂）。
  NotificationService.onReminderTapped = _openReminderFromNotification;

  // 冷启动：App 完全没运行时被通知拉起来，上面那个回调收不到这一次点击，
  // 必须主动查一次 —— 这是最容易漏的一条路径。
  final launchId = await NotificationService.instance.launchPayload();
  if (launchId != null) _pendingReminderId = launchId;

  runApp(const ProviderScope(child: PetApp()));

  // 崩溃记录自动补传。**放在 runApp 之后** ——
  // 上报要发网络请求，可能慢；放在它前面会拖慢首帧，
  // 而首帧延迟是用户能直接感知到的，崩溃上报不是。
  unawaited(_uploadCrashesLater());
}

/// 通知点击的统一入口。context 还没准备好就先记下来，首帧后再弹。
void _openReminderFromNotification(String reminderId) {
  final ctx = appNavigatorKey.currentContext;
  if (ctx == null) {
    _pendingReminderId = reminderId;
    return;
  }
```

```
  showReminderDueSheet(ctx, reminderId: reminderId);
}

/// 建一个本地用户。M6 接入账号后由登录流程取代。
///
/// 为什么现在就要：members 表和 created_by 字段都指向用户，
/// 没有用户记录时共养功能的种子数据接不上。
Future<void> _ensureLocalUser() async {
  final db = AppDatabase.instance.db;
  final rows =
      await db.query('users', where: 'id = ?', whereArgs: [kCurrentUserId]);
  if (rows.isNotEmpty) return;

  final now = DateTime.now().millisecondsSinceEpoch;
  await db.insert('users', {
    'id': kCurrentUserId,
    'nickname': L.isZh ? '我' : 'Me',
    'region': 'local',
    'created_at': now,
    'updated_at': now,
  });
}

class PetApp extends StatelessWidget {
  const PetApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: L.t('app.title'),
      debugShowCheckedModeBanner: false,
      navigatorKey: appNavigatorKey,
      theme: buildAppTheme(),
      home: const HomeShell(),
    );
  }
}

/// 构建主题。所有取值来自 core/theme.dart，此处不写裸色值。
ThemeData buildAppTheme() {
  final scheme = ColorScheme.fromSeed(
    seedColor: AppColors.seed,
    brightness: Brightness.light,
  ).copyWith(
    primary: AppColors.primary,
    surface: AppColors.surface,
    outlineVariant: AppColors.border,
    onSurface: AppColors.textPrimary,
    onSurfaceVariant: AppColors.textSecondary,
    outline: AppColors.textTertiary,
```

```
);

return ThemeData(
  colorScheme: scheme,
  useMaterial3: true,
  scaffoldBackgroundColor: AppColors.pageBg,
  textTheme: const TextTheme(
    headlineSmall: AppText.hero,
    titleLarge: AppText.pageTitle,
    titleMedium: AppText.section,
    titleSmall: AppText.section,
    bodyLarge: AppText.body,
    bodyMedium: AppText.body,
    bodySmall: AppText.caption,
    labelLarge: TextStyle(fontSize: 14, fontWeight: FontWeight.w600),
    labelMedium: AppText.caption,
    labelSmall: AppText.caption,
  ),
  splashFactory: InkSparkle.splashFactory,
  cardTheme: CardThemeData(
    elevation: 0,
    margin: EdgeInsets.zero,
    color: AppColors.surface,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(AppRadius.card),
      side: const BorderSide(color: AppColors.border),
    ),
  ),
  appBarTheme: const AppBarTheme(
    backgroundColor: AppColors.pageBg,
    surfaceTintColor: Colors.transparent,
    elevation: 0,
    centerTitle: false,
    titleTextStyle: TextStyle(
      fontSize: 17,
      fontWeight: FontWeight.w600,
      color: AppColors.textPrimary,
    ),
  ),
  navigationBarTheme: NavigationBarThemeData(
    backgroundColor: Colors.white,
    surfaceTintColor: Colors.transparent,
    indicatorColor: Colors.transparent,
    // 参考稿的底部栏没有胶囊底，靠图标与文字变色表示选中。
    height: 62,
    elevation: 0,
    labelBehavior: NavigationDestinationLabelBehavior.alwaysShow,
    labelTextStyle: WidgetStateProperty.resolveWith((states) {
      final selected = states.contains(WidgetState.selected);
      return TextStyle(
```

```
      fontSize: 12,
      fontWeight: selected ? FontWeight.w600 : FontWeight.w400,
      color: selected ? AppColors.primaryText : AppColors.textSecondary,
    );
  }),
  iconTheme: WidgetStateProperty.resolveWith((states) {
    final selected = states.contains(WidgetState.selected);
    return IconThemeData(
      size: 23,
      color: selected ? AppColors.primaryText : AppColors.textSecondary,
    );
  }),
),
dividerTheme: const DividerThemeData(
  color: AppColors.divider,
  thickness: 1,
  space: 1,
),
filledButtonTheme: FilledButtonThemeData(
  style: FilledButton.styleFrom(
    backgroundColor: AppColors.primary,
    foregroundColor: Colors.white,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(AppRadius.chip),
    ),
    minimumSize: const Size(0, AppSpace.tapTarget),
    padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 12),
    textStyle: const TextStyle(fontSize: 14, fontWeight: FontWeight.w600),
  ),
),
textButtonTheme: TextButtonThemeData(
    style: TextButton.styleFrom(
  foregroundColor: AppColors.primaryText,
  minimumSize: const Size(0, AppSpace.tapTarget),
  textStyle: const TextStyle(fontSize: 14, fontWeight: FontWeight.w600),
)),
chipTheme: ChipThemeData(
  backgroundColor: AppColors.surface,
  selectedColor: AppColors.primaryLight,
  side: const BorderSide(color: AppColors.border),
  // 两处 color 都必须显式给，否则选中态的文字会看不清。
  //
  // ChoiceChip 选中的文字取的是 `secondaryLabelStyle`，不是 `labelStyle`
  //  （见 SDK choice_chip.dart：`labelStyle ?? (selected ? chipTheme.
  // secondaryLabelStyle : null)`）。原来只写了 `TextStyle(fontSize: 12.5)`
  // —— 既漏了 `secondaryLabelStyle`，`labelStyle` 也没给颜色，于是选中态
  // 的文字颜色由 ColorScheme 推导，落到浅色上；白字压 `primaryLight`
  // (#EDE9FE) 的浅紫底，基本看不见（实测：记一笔弹窗里选中的「用药」
  // 「口服」看不清）。
  labelStyle: const TextStyle(
```

```
        fontSize: 12.5,
        color: AppColors.textSecondary,
      ),
      secondaryLabelStyle: const TextStyle(
        fontSize: 12.5,
        color: AppColors.primary,
        fontWeight: FontWeight.w500,
      ),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(AppRadius.chip),
      ),
    ),
    inputDecorationTheme: InputDecorationTheme(
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
        borderSide: const BorderSide(color: AppColors.border),
      ),
      enabledBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
        borderSide: const BorderSide(color: AppColors.border),
      ),
      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
        borderSide: const BorderSide(color: AppColors.primary, width: 1.5),
      ),
      labelStyle:
          const TextStyle(color: AppColors.textSecondary, fontSize: 13.5),
      floatingLabelStyle:
          const TextStyle(color: AppColors.primary, fontSize: 13),
    ),
    bottomSheetTheme: const BottomSheetThemeData(
      backgroundColor: AppColors.surface,
      surfaceTintColor: Colors.transparent,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
      ),
    ),
  );
}

class HomeShell extends StatefulWidget {
  const HomeShell({super.key});

  @override
  State<HomeShell> createState() => _HomeShellState();
}

class _HomeShellState extends State<HomeShell> {
  int _index = 0;
```

```
@override
void initState() {
  super.initState();

  // 同步是**手动**的：只在「我的」页点「立即同步」时跑，外加登录、
  // 接受邀请这类必要动作后各跑一次。
  //
  // 原来这里挂着一个 60 秒的 Timer.periodic 后台同步，外加回到前台
  // 无条件同步一次。用户反馈「一直在自己同步」，故整体去掉。代价是：
  // 本地记了东西如果不点同步，就一直只存在本机（「我的」页会显示
  // 「N 条待同步」）。自动同步要加回来的话，别退回「每分钟都打网络」。
  //
  // 启动后静默查一次更新。放在首帧之后，不占启动时间；
  // 检查失败或已是最新都**不打扰用户**（详见 ui/update_flow.dart）。
  WidgetsBinding.instance.addPostFrameCallback((_) {
    if (!mounted) return;
    runUpdateCheck(context, interactive: false);
    // Restore local alarm slots only; this does not initiate network sync.
    if (AppDatabase.instance.isOpen) {
      unawaited(ProviderScope.containerOf(context, listen: false)
          .read(appActionsProvider)
          .refreshNotifications());
    }

    // 冷启动时被通知点开：那时还没有 context，只能等首帧。
    final pending = _pendingReminderId;
    if (pending != null) {
      _pendingReminderId = null;
      showReminderDueSheet(context, reminderId: pending);
    }
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: SafeArea(
      bottom: false,
      child: IndexedStack(
        index: _index,
        children: const [
          TodayScreen(),
          RecordsScreen(),
          ProfileScreen(),
          MeScreen(),
        ],
      ),
    ),
    bottomNavigationBar: Container(
      decoration: const BoxDecoration(
```

```
          color: Colors.white,
          border: Border(top: BorderSide(color: AppColors.divider)),
        ),
        child: NavigationBar(
          selectedIndex: _index,
          onDestinationSelected: (i) => setState(() => _index = i),
          destinations: [
            NavigationDestination(
              icon: const Icon(Icons.home_outlined),
              selectedIcon: const Icon(Icons.home_rounded),
              label: L.t('tab.home'),
            ),
            NavigationDestination(
              icon: const Icon(Icons.edit_note_outlined),
              selectedIcon: const Icon(Icons.edit_note),
              label: L.t('tab.records'),
            ),
            NavigationDestination(
              icon: const Icon(Icons.pets_outlined),
              selectedIcon: const Icon(Icons.pets),
              label: L.t('tab.profile'),
            ),
            NavigationDestination(
              icon: const Icon(Icons.person_outline),
              selectedIcon: const Icon(Icons.person),
              label: L.t('tab.me'),
            ),
          ],
        ),
      ),
    );
  }
}

// ==================== lib/core/app_capabilities.dart ====================
/// 平台功能入口。地区规则仍由 region.dart 管理，数据层不读取功能开关。
library;

import 'dart:io';

import 'feature_flags.dart';

enum AppPlatform { android, ios, other }

enum AppFeature {
  petProfiles,
  familyCareBoard,
  medicationCourses,
  symptomObservations,
  careHandoff,
```

```
  localBackup,
  petDeletion,
  walkTracking,
  apkUpdates,
  iosWidgets,
  iosShortcuts,
}

class AppCapabilities {
  const AppCapabilities({required this.platform});

  final AppPlatform platform;

  static AppCapabilities get current => AppCapabilities(
        platform: Platform.isAndroid
            ? AppPlatform.android
            : Platform.isIOS
                ? AppPlatform.ios
                : AppPlatform.other,
      );

  bool supports(AppFeature feature) => switch (feature) {
        AppFeature.petProfiles ||
        AppFeature.familyCareBoard ||
        AppFeature.medicationCourses ||
        AppFeature.symptomObservations ||
        AppFeature.careHandoff ||
        AppFeature.localBackup ||
        AppFeature.petDeletion =>
          true,
        AppFeature.walkTracking =>
          kWalkEnabled && platform != AppPlatform.other,
        AppFeature.apkUpdates => platform == AppPlatform.android,
        // 预留苹果扩展；完成原生实现和真机验证后再开放入口。
        AppFeature.iosWidgets =>
          platform == AppPlatform.ios && kIosWidgetsEnabled,
        AppFeature.iosShortcuts =>
          platform == AppPlatform.ios && kIosShortcutsEnabled,
      };
}

// ==================== lib/core/feature_flags.dart ====================
/// 功能开关 —— 已经写好但还没到发布条件的模块，从这里统一关闭。
///
/// 用法约定：
/// - 开关是 `const bool`，让 Dart 的 tree-shake 把关掉的代码整块丢掉，
///   死代码不会进包体，也不需要担心「关了但还在跑」。
/// - 每个开关必须写清楚**为什么关**和**什么条件下打开**，
///   否则半年后没人敢动，就变成永久注释掉了。
/// - 只允许 UI 入口读开关；数据层（repository / migration）不要读。
```

```
///   数据库里已经存的数据不能因为开关来回跳。
library;

/// 遛狗：GPS 轨迹采集尚未接入。
///
/// **为什么关**：`geolocator` 目前零引用 —— 没有任何代码往 `walk_points`
/// 写轨迹点，`endSession` 用 `totalDistanceM(points)` 算出来的距离恒为 0。
/// 也就是说现在跑一趟遛狗，结果页永远显示 0.0 km，用户只会当成 bug。
///
/// **什么时候开**：第三批 Task 10。接入 `positionStream` → 批量写点 →
/// 50m 漂移过滤之后，把这里改成 true，同时补一条「距离非 0」的集成测试。
///
/// 关掉期间代码一行不删，schema / model / repository / 迁移测试全部保留。
const bool kWalkEnabled = false;

/// 苹果小组件：尚未实现 WidgetKit 扩展和共享数据容器。
/// 接入原生扩展、验证同步与隐私展示后才能打开。
const bool klosWidgetsEnabled = false;

/// 苹果快捷指令：尚未实现原生 App Intents 桥接。
/// 完成动作权限检查、重复执行保护和真机验证后才能打开。
const bool klosShortcutsEnabled = false;

// ==================== lib/core/l10n.dart ====================
/// 文本国际化。轻量实现：不引 flutter_localizations 的 gen-l10n，
/// 用区域 + 语言的简单查表，MVP 阶段够用且没有生成步骤。
library;

import 'region.dart';

/// 当前语言。中文区默认 zh，海外默认 en。
/// 后续接入系统 locale 时只改这一处。
enum AppLang { zh, en }

class L {
  L._();

  static const String _raw = String.fromEnvironment('LANG');

  static AppLang get current {
    if (_raw == 'zh' || _raw == 'en') {
      return _raw == 'zh' ? AppLang.zh : AppLang.en;
    }
    return AppRegion.isCn ? AppLang.zh : AppLang.en;
  }

  static bool get isZh => current == AppLang.zh;

  /// 取文案。[key] 找不到时回落到 key 本身，便于开发期暴露漏翻。
  static String t(String key) {
```

```
  final table = isZh ? _zh : _en;
  return table[key] ?? _en[key] ?? key;
}

/// 把异常渲染成**给用户看**的文案。
///
/// 分两类，这是关键：
///
/// 1. **领域异常**（`ArgumentError` / `StateError`）抛出的是翻译 key
///    （如 ``'care.error.changed'``），直接查表即可 —— 这类是「我们主动
///    告诉用户原因」，文案是写好的。
/// 2. **其它异常**一律**不把原文甩给用户**。
///
/// 第 2 条是 2026-10-11 改的（P2）。原先的兜底是 ``'$error'``，
/// 于是界面上会出现：
///
/// ```
/// SocketException: Connection refused (OS Error: Connection refused, errno = 111),
/// address = api.pet.weiyuantool.com, port = 44300
/// ```
///
/// 三个问题：用户看不懂；暴露内部域名与端口；同类异常每台设备文案都不一样，
/// 用户反馈时说不清、我们也搜不到。审查时发现有约 8 处直接 `Text('$e')`。
///
/// 兜底文案刻意做得**可行动**（「检查网络后重试」）而不是「未知错误」——
/// 后者等于没说，用户唯一能做的就是截图问人。
static String error(Object error) {
  final message = error is ArgumentError ? error.message
      : error is StateError ? error.message : null;
  if (message is String) {
    // 领域异常的 message 是 key；查不到表说明它不是 key（比如直接抛了
    // 中文句子），此时 t() 会原样返回，仍然合理。
    return t(message);
  }
  return t('error.generic');
}

/// 断网/超时类的异常文案（比 [error] 更具体，用在本机网络明显不可用时）。
static String get offline => t('error.offline');

/// 两张表的键差。给测试用的 —— 加中文忘加英文是最常见的翻车方式，
/// 而 [t] 的回落逻辑（zh 缺就去 en 找）会把它藏起来，运行期看不出来。
static ({Set<String> onlyZh, Set<String> onlyEn}) tableDiff() {
  final zh = _zh.keys.toSet();
  final en = _en.keys.toSet();
  return (onlyZh: zh.difference(en), onlyEn: en.difference(zh));
}

/// 简单的占位符替换：'今天要做 {n} 件事' + {'n': '3'}
static String tp(String key, Map<String, Object?> params) {
```

```
  var s = t(key);
  params.forEach((k, v) => s = s.replaceAll('{$k}', '${v ?? ''}'));
  return s;
}

static const Map<String, String> _zh = {
  "home.reminders.title": "全部提醒",
  "home.reminders.refresh": "刷新本地提醒",
  "home.reminders.empty": "添加提醒后，可在这里查看所有排期。",
  "home.reminders.hint": "包含所有宠物的已启用提醒。到期后可打开并确认完成。",
  "home.reminders.overdue": "已到期",
  "home.reminders.today": "今天稍后",
  "home.reminders.later": "后续排期",
  "care.historyTitle": "家庭照护记录",
  "care.collapse": "收起记录",
  "health.tools": "健康工具",
  "health.coursesHint": "管理用药时间与疗程",
  "health.coursesCount": "{count} 个已启用疗程",
  "health.symptomsCount": "近 7 天 {count} 条观察",
  "health.handoffHint": "为家人或临时照护者准备交接卡",
  "observation.extra": "补充信息",
  "observation.extraHint": "可选：关联就诊记录或用药疗程",
  "handoff.regenerate": "重新生成预览",
  "handoff.editContact": "编辑联系方式",
  "handoff.noContact": "尚未填写联系方式，可前往“我的”补充。",
  "handoff.zoomHint": "点击预览可放大查看，确认后再分享。",
  "handoff.template.feeding": "喂食",
  "handoff.template.walk": "散步",
  "handoff.template.emergency": "紧急联系",
  "handoff.templateText.feeding": "喂食：食物名称、每次分量、时间；需避免的食物：",
  "handoff.templateText.walk": "散步：时间、时长、牵引要求；如厕习惯：",
  "handoff.templateText.emergency": "紧急联系：照护联系人及电话；常用医院及电话：",
  "backup.preparing": "正在准备本地数据…",
  "backup.packing": "正在打包记录与附件…",
  "backup.verifying": "正在校验备份文件…",
  "backup.saving": "请在系统分享窗口选择保存位置。",
  "backup.selecting": "请选择备份 ZIP 文件。",
  "backup.awaitingConfirm": "校验完成，请确认恢复内容。",
  "backup.importing": "正在恢复为新的宠物副本…",
  "backup.petCount": "{count} 只宠物",
  "backup.recordCount": "{count} 条记录",
  "backup.attachmentCount": "{count} 条附件信息",
  "backup.neverGenerated": "尚未生成过本地备份",
  "backup.lastGenerated": "上次生成：{time}（请确认已保存文件）",
  "backup.pausedHint": "恢复后的提醒均已暂停。请先核对时间，再按需要启用，避免重复提醒。",
  "backup.reviewCourses": "检查恢复的用药疗程",

  "observation.title": "症状观察",
  "observation.add": "记一次观察",
  "observation.help": "记录你看到的变化；保存后可从记录详情添加照片或文档。",
```

"observation.empty": "还没有观察记录",
"observation.week": "最近 7 天：{count} 条观察记录",
"observation.symptom": "观察项目",
"observation.count": "发生次数（可不填，1－1000）",
"observation.countValue": "{count} 次",
"observation.appetite": "食欲",
"observation.energy": "精神状态",
"observation.stool": "排便状态",
"observation.medical": "关联就诊（可选）",
"observation.course": "关联疗程（可选）",
"observation.none": "不关联",
"observation.invalid": "请检查观察项目，次数须为 1－1000 的整数",
"observation.linkUnavailable": "关联条目已删除或不可用",
"observation.symptom.vomiting": "呕吐",
"observation.symptom.loose_stool": "腹泻／软便",
"observation.symptom.cough": "咳嗽",
"observation.symptom.itch": "抓挠／皮肤变化",
"observation.symptom.appetite": "食欲变化",
"observation.symptom.energy": "精神变化",
"observation.symptom.other": "其它观察",
"observation.state.unobserved": "未观察",
"observation.state.normal": "和平时一样",
"observation.state.reduced": "减少",
"observation.state.increased": "增加",
"observation.state.low": "没精神",
"observation.state.restless": "不安",
"observation.state.soft": "偏软",
"observation.state.liquid": "水样",
"observation.state.hard": "偏硬",
"handoff.title": "临时照护交接卡",
"handoff.help": "供寄养或临时照护者查看。先补充喂食时间、习惯和紧急联系信息，再预览并分享。",
"handoff.period": "照护期间",
"handoff.from": "开始",
"handoff.to": "结束",
"handoff.instructions": "交接说明",
"handoff.instructionsHint": "例如喂食时间、份量、外出习惯、医院电话",
"handoff.contact": "联系人",
"handoff.includeContact": "附上我的联系方式",
"handoff.latestFood": "最近一次喂食记录",
"handoff.tasks": "照护安排",
"handoff.reminder": "提醒",
"handoff.footer": "这是主人填写的照护资料快照，请按主人及兽医确认的安排执行。生成后修改的安排不会自动更
"handoff.preview": "生成预览",
"handoff.share": "分享交接卡图片",
"handoff.invalidDates": "结束日期不能早于开始日期",
"backup.title": "备份与恢复",
"backup.help": "将本机宠物档案、记录、疗程、费用、照片、文档和联系方式保存为 ZIP 文件。生成后请在分享面
"backup.privacy": "备份文件未加密，包含个人信息。请保存到你信任的位置；只有远程链接的附件会保留链接。",
"backup.export": "生成并保存备份",
"backup.restore": "从备份恢复",

"backup.confirm": "确认恢复内容",
"backup.preview": "备份日期：{date}\n宠物：{pets}\n记录：{records}\n本机文件：{files}",
"backup.restoreInfo": "恢复为独立宠物副本，保留原有数据。共养权限和登录状态需重新设置；恢复的提醒默认暂
"backup.contact": "同时恢复备份中的联系方式",
"backup.exported": "已生成 {pets} 只宠物、{files} 个文件的备份，请确认已在分享面板保存。",
"backup.restored": "已恢复 {count} 只宠物的数据。请核对副本并重新开启需要的提醒。",
"backup.invalid": "备份文件损坏、格式不支持或内容校验失败",
"backup.changed": "备份文件已变化，请重新选择并确认",
"backup.regionMismatch": "该备份属于另一地区，请在对应地区的 App 中恢复",
"backup.alreadyRestored": "这份备份已经恢复过，无需重复导入",
"backup.tooLarge": "备份超过 512 MB 或文件数量过多，请先整理附件",
"backup.missingFile": "有本机附件或头像文件缺失，未生成不完整备份。请检查相关资料后重试。",
"backup.noUser": "用户资料尚未准备好，请返回后重试",
"backup.busy": "备份或恢复正在进行，请稍候",

'care.title': '家庭照护看板',
'care.empty': '今天还没有照护记录',
'care.pending': '待完成',
'care.completed': '已完成 / 已记录',
'care.me': '你',
'care.other': '家庭成员',
'care.unknown': '未记录照护者',
'care.refresh': '更新家庭记录',
'care.syncHint': '共养记录通过手动同步更新；离线时显示本机记录。',
'care.lastSync': '上次同步：{time}',
'care.showAll': '查看全部（{count} 条）',
'care.alreadyDone': '这次照护已记录，请勿重复给药或喂食',
'care.error.changed': '这条待办已变更，请刷新后重试',
'care.readOnly': '你只有查看权限',
'care.noPending': '今天没有待完成的提醒',
'med.title': '用药疗程',
'med.add': '新增疗程',
'med.edit': '编辑疗程',
'med.empty': '还没有用药疗程',
'med.emptyHint': '填写已有医嘱的药名、剂量、日期和时间，逐次记录给药。',
'med.start': '开始日期',
'med.end': '结束日期',
'med.times': '每天给药时间',
'med.addTime': '添加时间',
'med.stock': '初始药品数量（可选）',
'med.stockHint': '数量和每次用量使用同一单位，如片数或毫升',
'med.units': '每次消耗数量',
'med.remaining': '估计剩余：{count}',
'med.lowStock': '药量不足下一次，请及时补充',
'med.next': '下次：{time}',
'med.active': '进行中',
'med.paused': '已暂停',
'med.finished': '已结束',
'med.pause': '暂停疗程',
'med.resume': '恢复疗程',

```
'med.pauseHint': '疗程有固定时间；如需暂停，请到用药疗程中操作。',
'med.given': '记录这次给药',
'med.givenConfirm': '确认已给药',
'med.givenHint': '{name} · {dose}\n计划时间：{time}\n只在实际给药后确认。',
'med.history': '给药记录（{count} 次）',
'med.noHistory': '还没有给药记录',
'med.saved': '疗程已保存',
'med.instructions': '按已有医嘱填写；这里用于记录和提醒。',
'med.error.required': '请填写药名和剂量',
'med.error.dates': '结束日期不能早于开始日期',
'med.error.times': '请选择 1–8 个不重复的给药时间',
'med.error.stock': '数量须为有效数字，每次用量须大于 0',
'med.error.expired': '疗程内已没有未来的给药时间，请调整日期',
'med.error.schedule': '疗程时间无效，请编辑后重试',
'pet.delete': '删除宠物',
'pet.delete.title': '删除 {name}？',
'pet.delete.hint': '宠物将从列表中移除，相关提醒停止；已有记录保留，不再显示在宠物列表中。同步后其他家
'pet.delete.ownerOnly': '只有主人可以删除宠物',
'pet.deleted': '宠物已删除',
'app.title': '我的宠物',
'tab.home': '首页',
'tab.today': '今日',
'tab.records': '记录',
'tab.profile': '档案',
'tab.me': '我的',

'home.greetingMorning': '早上好',
'home.greetingAfternoon': '下午好',
'home.greetingEvening': '晚上好',
'home.greeting.morning': '早上好',
'home.greeting.noon': '中午好',
'home.greeting.afternoon': '下午好',
'home.greeting.evening': '晚上好',
'home.subtitle': '{name} 今天一切都好',
'home.subtitleEmpty': '今天也是爱宠陪伴你的美好一天',
'home.lastRecord': '最近记录',
'home.lastRecord.today': '今天',
'home.lastRecord.todayAt': '今天 {time}',
'home.lastRecord.daysAgo': '{n} 天前',
'home.todoEmpty': '今天没有待办，放松一下',
'home.todoCaughtUp': '一切都在计划内',
'home.todo': '今日待办',
'home.switchPet': '切换宠物',
'home.viewAll': '查看全部',
'home.weekOverview': '本周概览',
'home.weekWalks': '遛狗',
'home.weekRecords': '记录',
'home.weekWeight': '体重',
'home.weekWeightCurrent': '当前体重',
'home.weekWalks.value': '{n} 次',
```

```
'home.weekRecords.value': '{n} 条',
'home.vsLast': '较上次',
'home.steady': '持平',

'today.title': '今日',
'today.empty.title': '今天没有待办',
'today.empty.hint': '给宠物加一条提醒，它会自动帮你排下次',
'today.upcoming': '接下来 7 天',
'due.tomorrow': '明天',
'due.inDays': '还有 {n} 天',
'due.overdueBy': '已过期 {n} 天',
'today.overdue': '已过期',
'today.done': '完成',
'today.doneToast': '已记录，下次：{next}',
'today.doneFinal': '已完成，这条提醒不会再来',
'today.snooze': '明天再说',
'today.startWalk': '开始遛狗',
'today.endWalk': '结束遛狗',
'today.walking': '遛狗中',
'today.quickAdd': '快捷记录',
'today.log': '今日记录',
'today.log.total': '共 {n} 条',
'today.quickAdd.more': '更多',

'records.title': '记录',
'records.empty.title': '还没有记录',
'records.empty.hint': '体重、疫苗、用药、喂食都可以记在这里',
'records.filter.all': '全部',
'records.weight.title': '体重趋势',
'records.weight.empty': '记两条以上体重就能看到曲线',
'records.noPet': '先添加一只宠物',
'records.search.hint': '搜索备注或类型',
'records.search.none': '没有匹配的记录',
'detail.recordedAt': '发生时间',
'detail.createdAt': '录入时间',
'detail.backfilledHint': '这条是事后补录的',
'detail.note': '备注',
'detail.change': '与上次相比',
'detail.photos': '照片',
'detail.photos.empty': '还没有照片。可补充药盒、处方、单据',
'detail.photos.camera': '拍照',
'detail.photos.gallery': '从相册选择',
'detail.photos.delete': '删除这张照片？',

// ---- 文档原件 ----
'detail.documents': '文档原件',
'detail.documents.empty': '还没有文档。化验单、疫苗本、保单都可以存这里',
'detail.documents.add': '添加文档',
'detail.documents.delete': '删除这份文档？',
'detail.documents.open': '打开',
```

```
'profile.section.documents': '文档原件',
'doc.pickFailed': '没选到文件',
'doc.pickCancelled': '已取消',
'doc.pickError': '打不开文件选择器：{e}',
'doc.readError': '读不到选中的文件：{e}',
'doc.tooLarge': '文件超过 {n} MB，换一个小一点的吧',
'doc.added': '已添加文档',
'doc.unknownType': '文件',
'doc.localOnly': '只存在本机，不会上传',
'detail.editTime': '改时间',
'detail.deleteConfirm': '删除这条记录？',
'detail.deleteConfirm.hint': '删掉后可以在数据导出里找回，但 App 内不再显示。',

'profile.title': '档案',
'profile.empty.title': '还没有宠物',
'profile.empty.hint': '添加后会自动生成疫苗和驱虫计划',
'profile.pet.senior': '老年',
'profile.tab.info': '资料',
'profile.tab.health': '健康',
'profile.tab.records': '记录',
'profile.tab.memory': '回忆',

// ---- 费用追踪 ----
'profile.tab.expense': '费用',
'expense.title': '费用追踪',
'expense.add': '记一笔花费',
'expense.thisMonth': '本月支出',
'expense.allTime': '累计',
'expense.avgMonth': '月均（有支出的月份）',
'expense.count': '{n} 笔',
'expense.trend': '近半年趋势',
'expense.byCategory': '本月花在哪',
'expense.recent': '最近支出',
'expense.month.short': '{m}月',
'expense.empty': '还没有记过花费',
'expense.empty.hint': '记下第一笔，就能看到每月花多少、都花在哪。',
'expense.amount': '金额',
'expense.amountRequired': '请填写大于 0 的金额',
'expense.category': '分类',
'expense.date': '日期',
'expense.note': '备注（可选）',
'expense.when.today': '今天',
'expense.save': '保存',
'expense.saved': '已记一笔',
'expense.delete': '删除',
'expense.deleted': '已删除',
'expense.category.food': '主粮零食',
'expense.category.medical': '就诊用药',
'expense.category.vaccine': '疫苗',
'expense.category.deworm': '驱虫',
```

```
'expense.category.grooming': '洗澡美容',
'expense.category.supply': '用品',
'expense.category.boarding': '寄养托运',
'expense.category.other': '其他',

'profile.section.basic': '基本信息',
'profile.section.family': '家庭成员',
'profile.section.traits': '个性特点',
'profile.addFamily': '添加家庭成员',
'profile.role.primary': '主要照顾者',
'profile.role.member': '家庭成员',
'profile.healthGood': '无待办',
'profile.healthAlert': '有待办',
'profile.shape.large': '大型犬',
'profile.shape.medium': '中型犬',
'profile.shape.small': '小型犬',
'profile.shape.cat': '猫',
'profile.field.gender': '性别',
'profile.gender.male': '公',
'profile.gender.female': '母',
'profile.gender.unknown': '未知',
'profile.neuteredSuffix': '已绝育',
'profile.field.shape': '体型',
'profile.field.color': '毛色',
'profile.edit': '编辑',
'profile.field.species': '种类',
'profile.field.breed': '品种',
'profile.field.birthday': '生日',
'profile.field.age': '年龄',
'profile.ageUnknown': '年龄未知',
'profile.field.weight': '体重',
'profile.field.traits': '个性特点',
'profile.traits.hint': '用来描述性格，例如友善、活泼',
'profile.traits.empty': '还没有填个性特点，点右上角「编辑」勾几个',
'profile.member.you': '你',
'profile.member.addSoon': '共养邀请在 M6 实现',
'profile.field.neutered': '绝育',
'profile.field.chip': '微芯片号',
'profile.field.allergy': '过敏',
'profile.section.walks': '遛狗记录',
'profile.walk.empty': '还没有遛狗记录',
'profile.reminders': '提醒计划',

'profile.section.quick': '快捷记录',
'profile.care.vaccine': '疫苗',
'profile.care.dewormInternal': '体内驱虫',
'profile.care.dewormExternal': '体外驱虫',
'profile.care.checkup': '体检与就诊',
'profile.care.grooming': '洗澡美容',
'profile.care.last': '上次 {v}',
```

```
'profile.care.lastNone': '还没有记录',
'profile.care.next': '下次 {v}',
'profile.care.unscheduled': '未排期',
'profile.care.off': '提醒已关',
'profile.care.log': '记一笔',

// ---- 导出报告（分享给兽医）----
'report.generatedAt': '生成于',
'report.section.profile': '宠物档案',
'report.section.care': '预防台账',
'report.section.weight': '体重趋势',
'report.section.records': '最近记录',
'report.empty': '还没有任何记录。',
'report.action': '导出健康报告',
'report.failed': '导出失败，请重试',
'report.disclaimer': '本报告由「我的宠物」按你的记录生成，属建议性内容，请以兽医意见为准。',
'profile.ledger.hint':
    '排期按生日和兽医常规自动推算。点右侧「记一笔」把实际做过的补上 —— 台账的「上次」就是这么来的。',

'profile.section.preventive': '预防保健',
'profile.preventive.empty': '还没有排期。填了生日就会自动生成疫苗和驱虫计划',
'profile.section.medical': '病史',
'profile.medical.empty': '还没有就诊记录',
'profile.allergy.empty': '还没有记录过敏',

'me.title': '我的',
'me.build.title': '当前构建',
'me.build.region': '市场区域',
'me.build.api': '服务地址',
'me.build.map': '地图渲染',
'me.build.geocoder': '地理编码',
'me.build.immunization': '建议排期规则集',
'me.build.icp': '备案号展示',
'me.build.locale': '语言',
'me.build.units': '默认单位',
'me.icp': '备案号将在此展示',
'me.about': '关于',
'me.about.body': '本应用提供的疫苗与驱虫排期为通用建议，不构成诊断或处方。'
    '不同个体的情况差异很大，请以执业兽医的意见为准。',
'me.disclaimer.short': '建议性内容，以兽医意见为准',
'me.section.pets': '我的宠物',
'me.section.data': '数据',
'me.setting.notify': '提醒通知',
'me.setting.units': '单位偏好',
'me.setting.export': '导出数据',
'me.setting.about': '关于与免责声明',

'action.add': '添加',
'action.save': '保存',
'action.cancel': '取消',
```

```
'action.delete': '删除',
'action.archive': '归档',
'action.retry': '重试',
'action.close': '关闭',
'action.back': '上一步',
'action.next': '下一步',

// 异常兜底（P2）。刻意写成「可行动」的句子而不是「未知错误」——
// 后者用户唯一能做的只有截图问人。
'error.generic': '操作没成功，请稍后重试。',
'error.offline': '网络不可用，请检查连接后重试。',

'addPet.progress': '第 {a} 步 / 共 {b} 步',
'addPet.step1.title': '先认识一下',
'addPet.step1.hint': '只需要名字，别的都可以以后再补',
'addPet.step2.title': '补一点资料',
'addPet.step2.hint': '填生日才能推算下次该什么时候打疫苗，其余随时可跳过',
'addPet.step3.title': '打过的疫苗',
'addPet.step3.hint': '按实际勾选，打过的不再重复排期',
'addPet.title': '添加宠物',
'addPet.name': '名字',
'addPet.name.hint': '例如：豆豆',
'addPet.species': '种类',
'addPet.species.dog': '狗',
'addPet.species.cat': '猫',
'addPet.species.other': '其他',
'addPet.breed': '品种（可选）',
'addPet.birthday': '生日（可选）',
'addPet.birthday.hint': '填了才会生成建议排期',
'addPet.birthday.estimated': '生日是估算的',
'addPet.submit': '创建并生成建议排期',
'addPet.nameRequired': '请填名字',
'addPet.created': '已创建 {n} 项提醒',

// 问卷式建档 —— 只问三个能改变输出的问题，不收集用不上的资料。
'addPet.region.cn': '中国大陆',
'addPet.region.intl': '海外',
'addPet.vaccine.none': '还没打过',
'addPet.vaccine.core': '打过联苗',
'addPet.vaccine.coreRabies': '联苗和狂犬都打过',

'addRecord.title': '记一笔',
'addRecord.type': '类型',
'addRecord.type.weight': '体重',
'addRecord.type.vaccine': '疫苗',
'addRecord.type.medication': '用药',
'addRecord.type.medical': '就诊',
'addRecord.type.grooming': '洗澡美容',
'addRecord.type.feeding': '喂食',
'addRecord.type.water': '饮水',
```

```
'addRecord.type.toilet': '排便',
'addRecord.type.sleep': '睡眠',
'addRecord.type.note': '笔记',
'addRecord.when': '什么时候',
'addRecord.when.now': '现在',
'addRecord.when.pick': '选择时间',
'addRecord.value': '数值',
'addRecord.note': '备注（可选）',
'addRecord.save': '保存记录',
'addRecord.saved': '已保存',

// 分类型表单 —— 体重/用药/喂食各有自己的字段，不再共用一个「数值」框。
'addRecord.weight.last': '上次 {v}',
'addRecord.med.name': '药品名',
'addRecord.med.nameHint': '例如：拜宠清、大宠爱',
'addRecord.med.nameRequired': '请填药品名',
'addRecord.med.dose': '剂量',
'addRecord.med.doseHint': '例如：1 片 / 0.5 ml',
'addRecord.med.route': '给药方式',
'addRecord.med.route.oral': '口服',
'addRecord.med.route.topical': '外用',
'addRecord.med.route.injection': '注射',
// 就诊记录：发给医生时用得上的四件事。
// 顺序按问诊顺序排（为什么来 → 在哪看 → 结论 → 怎么办），
// 用户照着医生问的填，不用在「数据结构」里跳来跳去。
'addRecord.med.complaint': '主诉（为什么来看）',
'addRecord.med.complaintHint': '例如：连续两天呕吐、不肯吃东西',
'addRecord.med.clinic': '医院 / 医生',
'addRecord.med.clinicHint': '例如：XX 宠物医院 · 王医生',
'addRecord.med.diagnosis': '诊断结论',
'addRecord.med.diagnosisHint': '例如：急性胃肠炎。先按医嘱观察两天',
'addRecord.med.advice': '医嘱',
'addRecord.med.adviceHint': '医生交代的注意事项、要不要复诊、饮食禁忌……',
'addRecord.med.oneRequired': '至少填一项，让以后的复诊有据可查',
'addRecord.feed.kind': '类型',
'addRecord.feed.kind.dry': '干粮',
'addRecord.feed.kind.wet': '湿粮',
'addRecord.feed.kind.treat': '零食',
'addRecord.feed.brand': '品牌（可选）',
'addRecord.feed.grams': '克数',

'timeline.today': '今天',
'timeline.yesterday': '昨天',
'timeline.daysAgo': '{n} 天前',
'timeline.backfilled': '补录',
// 从回忆相册加的照片，记录行上显示成「照片」而不是「笔记」
'record.kind.photo': '照片',

'walk.title': '遛狗',
'walk.start': '开始记录',
```

```
'walk.stop': '结束并保存',
'walk.points': '轨迹点',
'walk.running': '记录中…',
'walk.noMapHint': '当前市场不渲染地图底图，只记录轨迹坐标',
'walk.empty': '还没有记录',
'walk.result': '遛狗完成',
'walk.result.skip': '跳过',
'walk.mood.great': '很开心',
'walk.mood.good': '还不错',
'walk.mood.tired': '累了',
'walk.mood.anxious': '有点紧张',
'walk.mood.sick': '不太舒服',

'plan.vaccine.core': '核心疫苗',
'plan.vaccine.rabies': '狂犬疫苗',
'plan.deworm.internal': '体内驱虫',
'plan.deworm.external': '体外驱虫',
'plan.checkup.annual': '年度体检',
'plan.grooming': '洗澡美容',
'plan.checkup.senior': '老年体检',

// ---- M2.2 编辑档案 ----
'editPet.title': '编辑档案',
'editPet.section.basic': '基本信息',
'editPet.section.traits': '个性特点',
'editPet.section.health': '健康与识别',
'editPet.name': '名字',
'editPet.nameEmpty': '名字不能为空',
'editPet.species': '种类',
'editPet.breed': '品种',
'editPet.breedHint': '如：金毛、英短蓝猫',
'editPet.gender': '性别',
'editPet.birthday': '生日',
'editPet.birthdayPick': '选择生日',
'editPet.birthdayClear': '清除',
'editPet.birthdayEstimated': '生日是估算的',
'editPet.neutered': '已绝育',
// 通用的是 / 否。给「标签 + 值」这种布局用（如健康报告里的「绝育: 是」）。
'common.yes': '是',
'common.no': '否',
// 失败排查：侧载包在真机上跑，开发者不一定连着 Xcode，
// 拿不到控制台日志。让用户能自己把异常详情复制出来。
'common.copyDetail': '复制详情',
'common.detailCopied': '详情已复制，可直接发给我',
'editPet.chipNo': '芯片号',
'editPet.chipNoHint': '15 位数字，可留空',
'editPet.color': '毛色',
'editPet.colorHint': '如：奶油色、黑白',
'editPet.weightBaseline': '体重基线',
'editPet.weightBaselineHint': '成犬/成猫的常规体重，用于判断体型',
```

```
'editPet.allergy': '过敏',
'editPet.allergyHint': '如：鸡肉、花粉',
'editPet.note': '备注',
'editPet.traitsHint': '可多选，留空就不显示',
'editPet.save': '保存',
'editPet.saved': '已保存',
'editPet.kg': 'kg',

// ---- M2.2 头像 ----
'avatar.title': '宠物头像',
'avatar.camera': '拍照',
'avatar.gallery': '从相册选择',
'avatar.remove': '移除头像',
'avatar.failed': '头像保存失败，请重试',

// ---- 个性特点（存 code，显示查文案）----
'trait.friendly': '亲人',
'trait.playful': '爱玩',
'trait.calm': '安静',
'trait.social': '不怕生',
'trait.smart': '机灵',
'trait.foodie': '贪吃',
'trait.shy': '胆小',
'trait.active': '好动',

// ---- 自动更新 ----
'update.title': '发现新版本',
'update.current': '当前版本',
'update.latest': '最新版本',
'update.notes': '更新内容',
'update.now': '立即更新',
'update.later': '稍后',
'update.mustTitle': '需要更新后才能继续',
'update.mustBody': '这个版本太旧了，请更新后继续使用。',
'update.downloading': '正在下载 {percent}%',
'update.installing': '正在打开安装器…',
'update.failed': '下载失败，请检查网络后重试',
'update.upToDate': '已是最新版本',
'update.check': '检查更新',
'update.checkFailed': '检查更新失败，请稍后再试',
'update.permissionHint': '请在系统设置里允许本应用安装未知来源的应用',
'update.noApk': '这个版本暂不支持应用内更新，请到官网下载',
'me.version': '版本',
'me.privacy': '隐私政策',
'me.feedback': '问题反馈',
'feedback.hint': '遇到问题？把下面的内容发给我就行 —— '
    '里面已经自动带上了崩溃信息和 App 版本，不用你描述。',
'feedback.logs': '最近的异常记录',
'feedback.noLogs': '没有异常记录。如果是功能用不了也算，照样发给我。',
'feedback.clear': '清空',
```

```
'feedback.describe': '补充说明（可选）',
'feedback.describeHint': '比如：加了狗之后体重那一栏是空的',
'feedback.copy': '复制全部内容并发给我',
'feedback.copied': '已复制，粘贴到微信发给我就行',
'feedback.send': '直接发送给我',
'feedback.sending': '发送中…',
'feedback.sent': '已发送，谢谢！我会尽快看',
'feedback.sendFailed': '发送失败。可能是没网或服务器暂时有问题 —— '
    '你可以改用下面的「复制」，发微信给我。',
'feedback.sendFailedAuth': '发送失败（登录状态异常）。请改用下面的「复制」发给我。',
'feedback.nothingToSend': '没有可发送的内容。写一句描述，或者勾选上面的异常记录。',
'feedback.autoSummary': '（用户未填写描述，仅提交了异常记录）',
'feedback.cleared': '已清空',
'me.terms': '用户协议',

// ---- 档案页的记录 / 回忆页签 ----
'profile.records.empty': '还没有完成的条目',
'profile.records.emptyHint': '在记录页记一条，这里就会按时间排起来',
'profile.memory.empty': '还没有照片',
'profile.memory.emptyHint': '直接拍一张，或者从相册挑一张 —— 会存成一条带图的记录',
'profile.memory.count': '共 {n} 张',
'profile.memory.add': '添加照片',
'profile.memory.added': '已加入回忆',

// ---- M4 手动新建提醒 ----
'reminder.add': '添加提醒',
'reminder.fillPlan': '补全建议排期（{n} 项）',
'reminder.fillPlanDone': '已补全 {n} 项',
'reminder.new': '新建提醒',
'reminder.edit': '编辑提醒',
'reminder.type': '类型',
'reminder.name': '名称',
'reminder.nameHint': '留空就用类型名',
'reminder.repeat': '重复',
'reminder.repeat.once': '只提醒一次',
'reminder.repeat.custom': '自定义天数',
'reminder.repeat.monthly': '每月',
'reminder.repeat.quarterly': '每 3 个月',
'reminder.repeat.halfYearly': '每 6 个月',
'reminder.repeat.yearly': '每年',
'reminder.repeat.everyNDays': '每 {n} 天',
'reminder.days': '间隔天数',
'reminder.daysHint': '两次提醒之间隔多少天',
'reminder.firstAt': '下次提醒',
'reminder.firstAtHint': '从这天开始提醒',
'reminder.save': '保存',
'reminder.saved': '已保存',
'reminder.deleted': '已删除',
'reminder.delete': '删除提醒',
'reminder.deleteConfirm': '删除这条提醒？',
```

```
'reminder.deleteConfirm.hint': '删除后不再提醒。已完成的记录会保留，不受影响。',
'reminder.source.auto': '按免疫规程自动生成',
'reminder.source.manual': '手动添加',
'reminder.due.unknown': '这条提醒已经不存在了',
'reminder.type.vaccine': '疫苗',
'reminder.type.deworm_internal': '体内驱虫',
'reminder.type.deworm_external': '体外驱虫',
'reminder.type.checkup': '体检',
'reminder.type.medication': '用药',
'reminder.type.other': '其他',

// ---- M5 联系方式 ----
'contact.title': '联系方式',
'contact.hint': '走失协查卡片会把它印在卡片上，方便拾到的人联系你',
'contact.loginLocked': '账号手机号和邮箱需要验证后才能换绑；其它联系方式可在下方填写',
'contact.phone': '手机号',
'contact.email': '邮箱',
'contact.wechat': '微信号',
'contact.note': '其它',
'contact.noteHint': '如：小区 3 栋王阿姨',
'contact.empty': '还没填联系方式',
'contact.save': '保存',
'contact.saved': '已保存',
'contact.missing': '填一个联系方式，走失卡片才有用',

// ---- M5 走失协查卡片 ----
'lost.title': '走失协查卡片',
'lost.action': '生成走失协查卡片',
'lost.preview': '这是别人看到的样子',
'lost.share': '分享',
'lost.photoHint': '加张照片，别人更容易认出来',
'lost.age': '年龄',
'lost.lastSeen': '最后出现',
'lost.unknownWhere': '地点未知',
'lost.since': '已走失 {n} 天',
'lost.contactMe': '看到请联系',
'lost.noContact': '（未填联系方式，建议先到「我的」补充）',
'lost.shareSubject': '{name} 的走失协查卡片',
'lost.failed': '生成图片失败，请重试',

// ---- M5 遛狗详情 ----
'walk.detail': '遛狗详情',
'walk.track': '轨迹',
'walk.noTrack': '这次没有记录到轨迹点',
'walk.distance': '距离',
'walk.duration': '时长',
'walk.mood': '心情',
'walk.shareText': '{name} 这次走了 {distance}，用时 {duration}',

// ---- M6 登录 ----
```

```
'auth.title': '登录账号',
'auth.why': '登录后可以在多台设备之间同步，也能和家人一起记录同一只宠物',
'auth.channel.sms': '手机号',
'auth.channel.email': '邮箱',
'auth.unifiedHint': '用手机号登录。这个手机号同时是你的出岫账号'
    '（官网的 ERP 记账、商城、AI 修图共用同一个），只记一个就行。',
'auth.target.phone': '手机号',
'auth.target.email': '邮箱地址',
'auth.target.required': '请填写手机号或邮箱',
'auth.sendCode': '获取验证码',
'auth.resendIn': '{n} 秒后可重发',
'auth.code': '验证码',
'auth.codeHint': '6 位数字',
'auth.codeRequired': '请填写 6 位验证码',
'auth.devCode': '开发模式验证码：{code}',
'auth.submit': '登录',
'auth.sent': '验证码已发送',
'auth.failed': '登录失败，请重试',
'auth.notLoggedIn': '未登录',
'auth.notLoggedInHint': '登录后数据会在多台设备之间同步',
'auth.account': '账号',
'auth.logout': '退出登录',
'auth.logoutConfirm': '退出登录？',
'auth.logoutHint': '本地数据会保留，重新登录同一个账号就能接着用。',
'auth.loggedOut': '已退出登录',

// ---- M6 同步状态 ----
'sync.title': '数据同步',
'sync.now': '立即同步',
'sync.never': '还没同步过',
'sync.lastAt': '上次同步 {time}',
'sync.pending': '{n} 项待上传',
'sync.upToDate': '已是最新',
'sync.syncing': '同步中…',
'sync.failed': '同步失败',
'sync.done': '同步完成',

// ---- M6 共养 ----
'members.invite': '邀请成员',
'members.inviteHint': '对方需要先注册过。填他的手机号或邮箱。',
'members.target': '手机号或邮箱',
'members.role': '权限',
'members.role.owner': '主人 · 全部权限',
'members.role.editor': '协作者 · 可记录，不能管成员',
'members.role.viewer': '仅查看',
'members.status.pending': '待接受',
'members.invite.sent': '邀请已发出',
'members.invite.failed': '邀请失败',
'members.alreadyMember': '对方已经是这只宠物的成员了',
'members.notRegistered': '对方还没有账号，让他先安装并登录一次',
```

```
  'members.remove': '移除',
  'members.removeConfirm': '把 {name} 移出共养？',
  'members.removeHint': '移除后他不能再看到这只宠物的数据，历史记录保留。',
  'members.removed': '已移除',
  'members.needLogin': '共养要先登录账号',
  'members.loadFailed': '成员列表拉取失败',
  'members.invites.title': '收到的邀请',
  'members.invites.accept': '接受',
  'members.invites.accepted': '已接受，正在同步',
  'members.me': '我',

  // ---- 密码登录 ----
  'auth.password': '密码',
  'auth.passwordHint': '至少 6 位',
  'auth.passwordRequired': '请输入密码',
  'auth.passwordTooShort': '密码至少 6 位',
  'auth.switchToCode': '用验证码登录',
  'auth.switchToPassword': '用密码登录',
  'auth.forgotPassword': '忘记密码？用验证码登录，登录后可重设',
  'auth.passwordFailed': '手机号或密码不对',
  'auth.setPassword.title': '设个密码，下次免短信',
  'auth.setPassword.hint': '以后就能用「手机号 + 密码」直接登录，不用再等验证码。',
  'auth.setPassword.placeholder': '输入密码',
  'auth.setPassword.confirm': '确认密码',
  'auth.setPassword.mismatch': '两次输入的密码不一致',
  'auth.setPassword.skip': '先跳过',
  'auth.setPassword.save': '保存',
  'auth.setPassword.done': '密码已设置',
};

static const Map<String, String> _en = {
  "home.reminders.title": "All reminders",
  "home.reminders.refresh": "Refresh local reminders",
  "home.reminders.empty": "Add a reminder to see all schedules here.",
  "home.reminders.hint": "Enabled reminders for all pets. Open a due reminder to confirm completion.",
  "home.reminders.overdue": "Due now",
  "home.reminders.today": "Later today",
  "home.reminders.later": "Upcoming schedules",
  "care.historyTitle": "Family care history",
  "care.collapse": "Show less",
  "health.tools": "Health tools",
  "health.coursesHint": "Manage medication schedules",
  "health.coursesCount": "{count} enabled courses",
  "health.symptomsCount": "{count} observations in the past 7 days",
  "health.handoffHint": "Prepare instructions for family or a pet sitter",
  "observation.extra": "Additional information",
  "observation.extraHint": "Optional: link a vet visit or medication course",
  "handoff.regenerate": "Regenerate preview",
  "handoff.editContact": "Edit contact details",
  "handoff.noContact": "No contact details yet. Add them in Me.",
```

"handoff.zoomHint": "Tap to zoom and check the card before sharing.",
"handoff.template.feeding": "Feeding",
"handoff.template.walk": "Walks",
"handoff.template.emergency": "Emergency contact",
"handoff.templateText.feeding": "Feeding: food, amount per meal, times; foods to avoid:",
"handoff.templateText.walk": "Walks: times, duration, leash instructions; toilet routine:",
"handoff.templateText.emergency": "Emergency: caregiver name and phone; usual vet and phone:",
"backup.preparing": "Preparing local data…",
"backup.packing": "Packing records and attachments…",
"backup.verifying": "Verifying the backup…",
"backup.saving": "Choose a save destination in the system share sheet.",
"backup.selecting": "Select a backup ZIP file.",
"backup.awaitingConfirm": "Verified. Please confirm what to restore.",
"backup.importing": "Restoring new pet copies…",
"backup.petCount": "{count} pets",
"backup.recordCount": "{count} records",
"backup.attachmentCount": "{count} attachment entries",
"backup.neverGenerated": "No local backup generated yet",
"backup.lastGenerated": "Last generated: {time} (confirm the file was saved)",
"backup.pausedHint": "Restored reminders are paused. Check their times before enabling them to avoid du
"backup.reviewCourses": "Review restored medication courses",

"observation.title": "Symptom observations",
"observation.add": "Add observation",
"observation.help": "Record what you observe. Add photos or documents from the saved record details.",
"observation.empty": "No observations yet",
"observation.week": "Last 7 days: {count} observations",
"observation.symptom": "Observation",
"observation.count": "Occurrences (optional, 1–1000)",
"observation.countValue": "{count} occurrences",
"observation.appetite": "Appetite",
"observation.energy": "Energy",
"observation.stool": "Stool",
"observation.medical": "Related vet visit (optional)",
"observation.course": "Related medication course (optional)",
"observation.none": "None",
"observation.invalid": "Check the observation. Occurrences must be an integer from 1 to 1000.",
"observation.linkUnavailable": "Related item is unavailable",
"observation.symptom.vomiting": "Vomiting",
"observation.symptom.loose_stool": "Diarrhea / soft stool",
"observation.symptom.cough": "Coughing",
"observation.symptom.itch": "Scratching / skin changes",
"observation.symptom.appetite": "Appetite change",
"observation.symptom.energy": "Energy change",
"observation.symptom.other": "Other observation",
"observation.state.unobserved": "Not observed",
"observation.state.normal": "Usual",
"observation.state.reduced": "Reduced",
"observation.state.increased": "Increased",
"observation.state.low": "Low energy",

```
    "observation.state.restless": "Restless",
    "observation.state.soft": "Soft",
    "observation.state.liquid": "Watery",
    "observation.state.hard": "Hard",
    "handoff.title": "Care handoff card",
    "handoff.help": "For a sitter or temporary caregiver. Add feeding times, routines and emergency contact
    "handoff.period": "Care period",
    "handoff.from": "From",
    "handoff.to": "To",
    "handoff.instructions": "Care instructions",
    "handoff.instructionsHint": "Feeding times, portions, routines and vet phone",
    "handoff.contact": "Contact",
    "handoff.includeContact": "Include my contact details",
    "handoff.latestFood": "Latest feeding record",
    "handoff.tasks": "Care schedule",
    "handoff.reminder": "Reminder",
    "handoff.footer": "Care information supplied by the owner. Follow the schedule confirmed with the owner
    "handoff.preview": "Generate preview",
    "handoff.share": "Share card image",
    "handoff.invalidDates": "End date must not precede start date",
    "backup.title": "Backup and restore",
    "backup.help": "Save local pet profiles, records, medication courses, expenses, photos, documents and c
    "backup.privacy": "The backup is not encrypted and contains personal information. Store it somewhere yo
    "backup.export": "Create and save backup",
    "backup.restore": "Restore from backup",
    "backup.confirm": "Review restore",
    "backup.preview": "Backup date: {date}\nPets: {pets}\nRecords: {records}\nLocal files: {files}",
    "backup.restoreInfo": "Restore independent pet copies while keeping existing data. Sharing permissions
    "backup.contact": "Also restore contact details",
    "backup.exported": "Created a backup of {pets} pets and {files} files. Make sure you saved it from the
    "backup.restored": "Restored data for {count} pets. Review the copies and enable the reminders you need
    "backup.invalid": "The backup is damaged, unsupported or failed validation",
    "backup.changed": "The backup changed. Select it again and review.",
    "backup.regionMismatch": "This backup belongs to another region. Restore it in the matching app.",
    "backup.alreadyRestored": "This backup has already been restored",
    "backup.tooLarge": "The backup exceeds 512 MB or contains too many files. Review your attachments first
    "backup.missingFile": "A local attachment or avatar is missing. Check your files before creating a comp
    "backup.noUser": "Your profile is not ready. Go back and try again.",
    "backup.busy": "A backup or restore is already running",

    'care.title': 'Family care',
    'care.empty': 'No care recorded today',
    'care.pending': 'Pending',
    'care.completed': 'Completed / recorded',
    'care.me': 'You',
    'care.other': 'Family member',
    'care.unknown': 'Caregiver not recorded',
    'care.refresh': 'Refresh family records',
    'care.syncHint': 'Shared care updates with manual sync. Offline, this shows local records.',
// ……（中间省略 25280 行，程序总量 28128 行）……
```

```
        'medical' => RecordType.medical,
        'feeding' => RecordType.feeding,
        _ => RecordType.note,
      };
}

// ==================== lib/ui/update_flow.dart ====================
/// 自动更新的界面流程。
///
/// 一个入口函数 [runUpdateCheck] 管两种场景：
/// - `interactive: false`（启动时静默检查）：只有当真的有新版才弹框，
///   检查失败/已是最新**什么都不做** —— 启动路径上不能有任何打扰。
/// - `interactive: true`（「我的」页手动点）：无论结果如何都给一句反馈，
///   用户主动问了就得知答案。
///
/// 强制更新（本机 build < 清单 minBuild）没有「稍后」，只能更新或退出应用。
library;

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:package_info_plus/package_info_plus.dart';

import '../core/l10n.dart';
import '../core/theme.dart';
import '../providers.dart';
import '../services/app_update_service.dart';

/// 检查并处理更新。
Future<void> runUpdateCheck(
  BuildContext context, {
  required bool interactive,
}) async {
  final container = ProviderScope.containerOf(context, listen: false);
  final service = container.read(appUpdateServiceProvider);

  // iOS 不能用安卓的版本清单、强制更新门槛和安装流程。
  if (!service.supportsUpdates) return;

  final info = await PackageInfo.fromPlatform();
  final currentBuild = int.tryParse(info.buildNumber) ?? 0;

  final outcome = await service.check(
    currentBuild: currentBuild,
    currentVersion: info.version,
  );
  if (!context.mounted) return;

  switch (outcome.status) {
    case UpdateCheckStatus.hasUpdate:
      // 平时只显示版本名（0.1.3）—— `0.1.3+5` 那种写法是给 pubspec 看的，
```

```
      // 用户看到会困惑。但**两个版本名恰好相同时**（出包用了 -NoBumpName，
      // 只抬 build 不抬名）必须补上 build，否则弹窗会显示成
      // 「当前 0.1.3 → 最新 0.1.3」，看起来像没得更新。
      final sameName = outcome.result!.manifest.version == info.version;
      await _showUpdateDialog(
        context,
        result: outcome.result!,
        service: service,
        currentVersion:
            sameName ? '${info.version} (+$currentBuild)' : info.version,
      );
    case UpdateCheckStatus.upToDate:
      if (interactive) _toast(context, L.t('update.upToDate'));
    case UpdateCheckStatus.unsupported:
      if (interactive) _toast(context, L.t('update.noApk'));
    case UpdateCheckStatus.failed:
      // 静默检查失败不打扰用户；手动检查要说一句，否则像是点了没反应。
      if (interactive) _toast(context, L.t('update.checkFailed'));
  }
}

Future<void> _showUpdateDialog(
  BuildContext context, {
  required UpdateCheckResult result,
  required AppUpdateService service,
  required String currentVersion,
}) async {
  final manifest = result.manifest;

  final go = await showDialog<bool>(
    context: context,
    barrierDismissible: !result.forced,
    builder: (ctx) => PopScope(
      // 强制更新时屏蔽返回键，否则用户可以绕过。
      canPop: !result.forced,
      child: AlertDialog(
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(AppRadius.card),
        ),
        title: Text(L.t(result.forced ? 'update.mustTitle' : 'update.title')),
        content: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            if (result.forced) ...[
              Text(
                L.t('update.mustBody'),
                style: const TextStyle(
                  fontSize: 13,
                  height: 1.55,
```

```
                color: AppColors.textSecondary,
              ),
            ),
            const SizedBox(height: AppSpace.gapM),
          ],
          _kv(L.t('update.current'), currentVersion),
          const SizedBox(height: 4),
          _kv(L.t('update.latest'), manifest.version, highlight: true),
          if ((manifest.notes ?? '').isNotEmpty) ...[
            const SizedBox(height: AppSpace.gapM),
            Text(
              L.t('update.notes'),
              style: const TextStyle(
                fontSize: 12.5,
                fontWeight: FontWeight.w600,
                color: AppColors.textPrimary,
              ),
            ),
            const SizedBox(height: 4),
            Text(
              manifest.notes!,
              style: const TextStyle(
                fontSize: 12.5,
                height: 1.6,
                color: AppColors.textSecondary,
              ),
            ),
          ],
        ],
      ),
      actions: [
        if (!result.forced)
          TextButton(
            onPressed: () => Navigator.pop(ctx, false),
            child: Text(L.t('update.later')),
          ),
        FilledButton(
          onPressed: () => Navigator.pop(ctx, true),
          child: Text(L.t('update.now')),
        ),
      ],
    ),
  ),
);

if (go != true || !context.mounted) return;
await _downloadAndInstall(context, service: service, manifest: manifest);
}

Future<void> _downloadAndInstall(
```

```
  BuildContext context, {
  required AppUpdateService service,
  required UpdateManifest manifest,
}) async {
  final progress = ValueNotifier<double?>(0);
  final status =
      ValueNotifier<String>(L.tp('update.downloading', {'percent': 0}));

  // 用不可关的对话框承载进度。下载中途被划掉会留下半个 APK，
  // 下次安装直接失败 —— 不如不让关。
  showDialog<void>(
    context: context,
    barrierDismissible: false,
    builder: (_) => PopScope(
      canPop: false,
      child: AlertDialog(
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(AppRadius.card),
        ),
        content: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            ValueListenableBuilder<String>(
              valueListenable: status,
              builder: (_, text, __) => Text(
                text,
                textAlign: TextAlign.center,
                style: const TextStyle(fontSize: 13.5),
              ),
            ),
            const SizedBox(height: AppSpace.gapL),
            ValueListenableBuilder<double?>(
              valueListenable: progress,
              builder: (_, value, __) => LinearProgressIndicator(
                value: value,
                minHeight: 6,
                borderRadius: BorderRadius.circular(3),
                backgroundColor: AppColors.divider,
              ),
            ),
          ],
        ),
      ),
    ),
  );

  var failed = false;
  var needsPermission = false;
  try {
    final path = await service.download(
```

```
      manifest,
      onProgress: (v) {
        progress.value = v;
        status.value = v == null
            ? L.t('update.downloading').replaceAll('{percent}', '…')
            : L.tp('update.downloading', {'percent': (v * 100).round()});
      },
    );
    status.value = L.t('update.installing');
    final installed = await service.installApk(path);
    if (!installed) needsPermission = true;
  } catch (_) {
    failed = true;
  }

  if (!context.mounted) return;
  // 关掉进度框。用 rootNavigator，避免被页内嵌套 Navigator 吃掉。
  Navigator.of(context, rootNavigator: true).pop();

  if (failed) {
    _toast(context, L.t('update.failed'));
    return;
  }
  if (needsPermission && context.mounted) {
    _toast(context, L.t('update.permissionHint'));
    // 顺手把用户送到授权页，省得自己去翻设置。
    await service.openInstallPermissionSettings();
  }
}

Widget _kv(String label, String value, {bool highlight = false}) {
  return Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Text(
        label,
        style: const TextStyle(fontSize: 12.5, color: AppColors.textSecondary),
      ),
      Text(
        value,
        style: TextStyle(
          fontSize: 12.5,
          fontWeight: highlight ? FontWeight.w700 : FontWeight.w500,
          color: highlight ? AppColors.primary : AppColors.textPrimary,
        ),
      ),
    ],
  );
}
```

```
void _toast(BuildContext context, String text) {
  ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(text)));
}

// ==================== lib/ui/walk_detail.dart ====================
/// 遛狗详情（M5）：轨迹地图 + 统计 + 心情备注 + 分享。
///
/// 地图的合规分叉（见 core/region.dart）：**中国区不渲染底图**，只画自绘轨迹
/// 折线 —— 境内展示地图底图涉及测绘资质。所以下面 `<intl>` 分支才有 TileLayer，
/// 中文区那块换成一块中性底 + 坐标文字。
/// 两个分支画的是**同一份坐标**，切换区域只影响底图，不影响轨迹本身。
library;

import 'package:flutter/material.dart';
import 'package:flutter_map/flutter_map.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
// `hide Path`：latlong2 里也有个 `class Path<T extends LatLng>`（画线用的
// 路径容器），它和 dart:ui 的 `Path` 同名。不挡掉的话，下面 CustomPaint 里
// 的 `Path()` 会被解析成 latlong2 那个，`moveTo`/`lineTo` 全部找不到。
import 'package:latlong2/latlong.dart' hide Path;

import '../core/l10n.dart';
import '../core/region.dart';
import '../core/theme.dart';
import '../core/units.dart';
import '../data/models.dart';
import '../providers.dart';
import '../services/share_helper.dart';
import 'sheet_scaffold.dart';
import 'widgets.dart';

Future<void> showWalkDetailSheet(
  BuildContext context, {
  required WalkSession session,
  String? petName,
}) {
  return showModalBottomSheet<void>(
    context: context,
    isScrollControlled: true,
    useSafeArea: true,
    builder: (_) => _WalkDetailSheet(session: session, petName: petName),
  );
}

class _WalkDetailSheet extends ConsumerWidget {
  const _WalkDetailSheet({required this.session, this.petName});

  final WalkSession session;
  final String? petName;
```

```
@override
Widget build(BuildContext context, WidgetRef ref) {
  final points = ref.watch(walkPointsProvider(session.id));
  const region = AppRegion.current;
  final distUnit = Units.defaultDistanceUnit(region);

  return FractionallySizedBox(
    heightFactor: 0.88,
    child: SheetScaffold(
      title: L.t('walk.detail'),
      scrollable: false,
      action: IconButton(
        tooltip: L.t('lost.share'),
        onPressed: () => _share(context, distUnit),
        icon: const Icon(Icons.ios_share_rounded, size: 19),
      ),
      child: points.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => errorPlaceholder(e),
        data: (pts) => ListView(
          padding: const EdgeInsets.fromLTRB(
            AppSpace.page,
            AppSpace.gapL,
            AppSpace.page,
            AppSpace.gapXl,
          ),
          children: [
            _TrackMap(points: pts),
            if (!region.mapRenderingEnabled) ...[
              const SizedBox(height: AppSpace.gapS),
              Text(
                L.t('walk.noMapHint'),
                style: const TextStyle(
                  fontSize: 11.5,
                  color: AppColors.textTertiary,
                ),
              ),
            ],

            const SizedBox(height: AppSpace.gapL),
            _Stats(session: session, unit: distUnit),

            const SizedBox(height: AppSpace.gapL),
            if ((session.mood ?? '').isNotEmpty)
              _Row(
                icon: Icons.mood_rounded,
                label: L.t('walk.mood'),
                value:
                    '${walkMoodEmoji(session.mood!)} ${L.t('walk.mood.${session.mood}')}',
              ),
```

```
              if ((session.note ?? '').isNotEmpty)
                _Row(
                  icon: Icons.sticky_note_2_outlined,
                  label: L.t('detail.note'),
                  value: session.note!,
                ),
              _Row(
                icon: Icons.route_outlined,
                label: L.t('walk.track'),
                value: pts.isEmpty
                    ? L.t('walk.noTrack')
                    : L.tp('walk.points', {'n': pts.length}),
              ),
            ],
          ),
        ),
      ),
    );
  }

  Future<void> _share(BuildContext context, DistanceUnit unit) async {
    final duration = durationLabel(session.durationS);
    await shareText(
      text: L.tp('walk.shareText', {
        'name': petName ?? '',
        'distance': Units.formatDistance(session.distanceM, unit),
        'duration': duration,
      }),
      context: context,
    );
  }
}

/// 轨迹地图。
///
/// 两个区域分支画的都是同一份 [points]：
/// - 海外（mapRenderingEnabled）：flutter_map + OSM 瓦片 + 折线 + 起终点标记
/// - 中国区：不拉瓦片，只在地图坐标空间里画折线（CustomPaint）
///
/// 为什么中国区还要画：轨迹形状本身是用户自己的数据，不涉及底图资质。
class _TrackMap extends StatelessWidget {
  const _TrackMap({required this.points});

  final List<WalkPoint> points;

  @override
  Widget build(BuildContext context) {
    if (points.isEmpty) {
      return Container(
        height: 200,
```

```
    alignment: Alignment.center,
    decoration: BoxDecoration(
      color: AppColors.surface,
      borderRadius: AppRadius.cardBorder,
      border: Border.all(color: AppColors.border),
    ),
    child: Text(
      L.t('walk.noTrack'),
      style: const TextStyle(fontSize: 12.5, color: AppColors.textSecondary),
    ),
  );
}

final latLngs = [
  for (final p in points) LatLng(p.lat, p.lng),
];
final center = _center(points);

return ClipRRect(
  borderRadius: AppRadius.cardBorder,
  child: SizedBox(
    height: 240,
    child: AppRegion.current.mapRenderingEnabled
        ? FlutterMap(
            options: MapOptions(
              initialCenter: center,
              // 缩放写死 16：算「适配整条轨迹」的度数很绕，
              // 而用户看轨迹时放大一点反而更清楚。
              initialZoom: 16,
              interactionOptions: const InteractionOptions(
                flags: InteractiveFlag.pinchZoom | InteractiveFlag.drag,
              ),
            ),
            children: [
              TileLayer(
                urlTemplate:
                    'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
                userAgentPackageName: 'com.weiyuantool.pet',
              ),
              PolylineLayer(
                polylines: [
                  Polyline(
                    points: latLngs,
                    strokeWidth: 4,
                    color: AppColors.primary,
                  ),
                ],
              ),
              MarkerLayer(
                markers: [
```

```
                    Marker(
                      point: latLngs.first,
                      width: 22,
                      height: 22,
                      child: const _Dot(color: AppColors.success),
                    ),
                    Marker(
                      point: latLngs.last,
                      width: 22,
                      height: 22,
                      child: const _Dot(color: AppColors.danger),
                    ),
                  ],
                ),
              ],
            )
          : _PlainTrack(points: points),
      ),
    );
  }

  /// 轨迹中心：取首尾的中点。比算包围盒简单，视觉上也够用 ——
  /// 遛狗轨迹不会是 L 形绕半个城市。
  static LatLng _center(List<WalkPoint> pts) => LatLng(
        (pts.first.lat + pts.last.lat) / 2,
        (pts.first.lng + pts.last.lng) / 2,
      );
}

/// 中国区用的无底图轨迹图。把经纬度线性映射到画布，保持长宽比。
class _PlainTrack extends StatelessWidget {
  const _PlainTrack({required this.points});

  final List<WalkPoint> points;

  @override
  Widget build(BuildContext context) {
    return ColoredBox(
      color: AppColors.primaryLight,
      child: CustomPaint(
        painter: _TrackPainter(points: points),
        size: Size.infinite,
      ),
    );
  }
}

class _TrackPainter extends CustomPainter {
  const _TrackPainter({required this.points});
```

```
final List<WalkPoint> points;

@override
void paint(Canvas canvas, Size size) {
  var minLat = points.first.lat, maxLat = points.first.lat;
  var minLng = points.first.lng, maxLng = points.first.lng;
  for (final p in points) {
    minLat = p.lat < minLat ? p.lat : minLat;
    maxLat = p.lat > maxLat ? p.lat : maxLat;
    minLng = p.lng < minLng ? p.lng : minLng;
    maxLng = p.lng > maxLng ? p.lng : maxLng;
  }

  final spanLat = (maxLat - minLat).abs();
  final spanLng = (maxLng - minLng).abs();
  // 起终点重合（原地打转）时给一个极小跨度，避免除零把线画到无穷远。
  final safeLat = spanLat < 1e-6 ? 1e-6 : spanLat;
  final safeLng = spanLng < 1e-6 ? 1e-6 : spanLng;

  const pad = 24.0;
  final w = size.width - pad * 2;
  final h = size.height - pad * 2;
  // 等比缩放，否则轨迹会被拉成横长条，形状就失真了。
  final scale = (w / safeLng) < (h / safeLat) ? (w / safeLng) : (h / safeLat);
  final dx = (w - safeLng * scale) / 2 + pad;
  final dy = (h - safeLat * scale) / 2 + pad;

  final path = Path();
  for (var i = 0; i < points.length; i++) {
    final x = dx + (points[i].lng - minLng) * scale;
    // 纬度往上增大，屏幕 y 往下增大，所以取反。
    final y = dy + (maxLat - points[i].lat) * scale;
    if (i == 0) {
      path.moveTo(x, y);
    } else {
      path.lineTo(x, y);
    }
  }

  canvas.drawPath(
    path,
    Paint()
      ..color = AppColors.primary
      ..style = PaintingStyle.stroke
      ..strokeWidth = 3
      ..strokeCap = StrokeCap.round
      ..strokeJoin = StrokeJoin.round,
  );

  void dot(Offset o, Color c) => canvas.drawCircle(o, 5, Paint()..color = c);
```

```
    dot(
      Offset(dx + (points.first.lng - minLng) * scale,
          dy + (maxLat - points.first.lat) * scale),
      AppColors.success,
    );
    dot(
      Offset(dx + (points.last.lng - minLng) * scale,
          dy + (maxLat - points.last.lat) * scale),
      AppColors.danger,
    );
  }

  @override
  bool shouldRepaint(covariant _TrackPainter old) => old.points != points;
}

class _Dot extends StatelessWidget {
  const _Dot({required this.color});

  final Color color;

  @override
  Widget build(BuildContext context) => Container(
        decoration: BoxDecoration(
          color: color,
          shape: BoxShape.circle,
          border: Border.all(color: Colors.white, width: 2.5),
        ),
      );
}

class _Stats extends StatelessWidget {
  const _Stats({required this.session, required this.unit});

  final WalkSession session;
  final DistanceUnit unit;

  @override
  Widget build(BuildContext context) {
    final (h, m) = Units.splitDuration(session.durationS);
    final dur = h > 0 ? '${h}h ${m}m' : '${m}m';

    return Container(
      padding: const EdgeInsets.symmetric(vertical: AppSpace.gapL),
      decoration: BoxDecoration(
        color: AppColors.surface,
        borderRadius: AppRadius.cardBorder,
        border: Border.all(color: AppColors.border),
      ),
      child: Row(
```

```
        children: [
          Expanded(
            child: _StatCell(
              label: L.t('walk.distance'),
              value: Units.formatDistance(session.distanceM, unit),
            ),
          ),
          const SizedBox(
            height: 32,
            child: VerticalDivider(width: 1, color: AppColors.divider),
          ),
          Expanded(
            child: _StatCell(label: L.t('walk.duration'), value: dur),
          ),
        ],
      ),
    );
  }
}

class _StatCell extends StatelessWidget {
  const _StatCell({required this.label, required this.value});

  final String label;
  final String value;

  @override
  Widget build(BuildContext context) => Column(
        children: [
          Text(
            value,
            style: const TextStyle(
              fontSize: 18,
              fontWeight: FontWeight.w700,
              color: AppColors.textPrimary,
            ),
          ),
          const SizedBox(height: 2),
          Text(
            label,
            style: const TextStyle(
              fontSize: 11.5,
              color: AppColors.textSecondary,
            ),
          ),
        ],
      );
}

class _Row extends StatelessWidget {
```

```
  const _Row({required this.icon, required this.label, required this.value});

  final IconData icon;
  final String label;
  final String value;

  @override
  Widget build(BuildContext context) => Padding(
        padding: const EdgeInsets.only(bottom: AppSpace.gapM),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Icon(icon, size: 17, color: AppColors.textTertiary),
            const SizedBox(width: AppSpace.gapM),
            Text(
              '$label：',
              style: const TextStyle(
                fontSize: 12.5,
                color: AppColors.textSecondary,
              ),
            ),
            Expanded(
              child: Text(
                value,
                style: const TextStyle(
                  fontSize: 13,
                  color: AppColors.textPrimary,
                  height: 1.5,
                ),
              ),
            ),
          ],
        ),
      );
}

// ==================== lib/ui/widgets.dart ====================
/// 通用小组件。四个页面共用，避免各写一套导致视觉漂移。
library;

import 'dart:io';

import 'package:flutter/material.dart';

import '../core/l10n.dart';
import '../core/reminder_text.dart';
import '../core/theme.dart';
import '../core/units.dart';
import '../data/models.dart';
import '../domain/immunization.dart';
```

```
// 本文件自己也要用 pad2 / GenderMark 等 —— `export ... show` 只对
// **导入本文件的人**生效，不会把名字带进本文件的作用域，所以这里要真 import。
import 'sheet_scaffold.dart';

// 提醒文案的解析与类型表在 core 里（通知服务也要用），
// 这里再导出一次，免得所有界面文件都要多 import 一个。
export '../core/reminder_text.dart'
    show kManualReminderTypes, reminderTypeLabel;

// 错误展示的两个 helper 定义在 sheet_scaffold.dart，但用法遍布所有界面。
// 从这里再导出一次：绝大多数界面本来就 import 了 widgets.dart，
// 不必为了 `errorPlaceholder` 再补一行 import。
export 'sheet_scaffold.dart'
    show
        confirmAction,
        confirmDelete,
        errorPlaceholder,
        pad2,
        showErrorSnack,
        showErrorSnackVia;

// 类型 → 文案的纯映射搬去了 domain/labels.dart（那边不引 Flutter，
// 纯逻辑层如「导出报告」的内容组装也能用）。这里转发一次，界面文件不必改 import。
export '../domain/labels.dart'
    show
        careKindLabel,
        expenseCategoryLabel,
        feedKindLabel,
        fileSizeLabel,
        attachmentTitle,
        medRouteLabel,
        mimeOfExt,
        planTypeLabel,
        petAgeLabel,
        recordPayloadSummary,
        recordTypeLabel,
        recordValueLine;

/// 统计格之间那道竖线。今日页的主卡、本周概览、遛狗结果都在用它。
class StatDivider extends StatelessWidget {
  const StatDivider({super.key});

  @override
  Widget build(BuildContext context) => Container(
        width: 1,
        height: 34,
        color: AppColors.divider,
      );
}
```

```
/// 空态。四个页面都有「还没有数据」的场景，统一这一种表达。
class EmptyState extends StatelessWidget {
  const EmptyState({
    super.key,
    required this.icon,
    required this.title,
    this.hint,
    this.action,
  });

  final IconData icon;
  final String title;
  final String? hint;
  final Widget? action;

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(builder: (context, constraints) {
      return SingleChildScrollView(
        child: ConstrainedBox(
          constraints: BoxConstraints(
            minHeight: constraints.hasBoundedHeight ? constraints.maxHeight : 0,
          ),
          child: Padding(
            padding: const EdgeInsets.all(AppSpace.gapXl),
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Container(
                  width: 72,
                  height: 72,
                  decoration: const BoxDecoration(
                    color: AppColors.primaryLight,
                    shape: BoxShape.circle,
                  ),
                  child: Icon(icon, size: 34, color: AppColors.primary),
                ),
                const SizedBox(height: AppSpace.gapXl),
                Text(title,
                    textAlign: TextAlign.center, style: AppText.section),
                if (hint != null) ...[
                  const SizedBox(height: AppSpace.gapS),
                  Text(hint!,
                      textAlign: TextAlign.center, style: AppText.caption),
                ],
                if (action != null) ...[
                  const SizedBox(height: AppSpace.gapXl),
                  action!,
                ],
              ],
```

```
            ),
          ),
        ),
      );
    });
  }
}

/// 区域色标（左竖条）+ 内容，用于今日待办卡片。
class AccentCard extends StatelessWidget {
  const AccentCard({
    super.key,
    required this.child,
    this.accent,
    this.onTap,
  });

  final Widget child;
  final Color? accent;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final cs = Theme.of(context).colorScheme;
    final color = accent ?? cs.primary;

    return Card(
      clipBehavior: Clip.antiAlias,
      margin: const EdgeInsets.only(bottom: 10),
      child: InkWell(
        onTap: onTap,
        child: IntrinsicHeight(
          child: Row(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Container(width: 4, color: color),
              Expanded(
                child: Padding(
                  padding: const EdgeInsets.fromLTRB(14, 12, 12, 12),
                  child: child,
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

```
/// 章节标题。
class SectionHeader extends StatelessWidget {
  const SectionHeader(this.title, {super.key, this.trailing});

  final String title;
  final Widget? trailing;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.fromLTRB(4, AppSpace.gapM, 4, AppSpace.gapS),
      child: Row(
        children: [
          Expanded(child: Text(title, style: AppText.section)),
          // Codex 加的：标题与右侧 trailing 之间留出间距，
          // 否则长标题会和 trailing 贴在一起。
          const SizedBox(width: 8),
          if (trailing != null) trailing!,
        ],
      ),
    );
  }
}

/// 档案页的「标签 - 值」行。
///
/// 参考稿里标签靠左、值靠右，中间留白，行高比默认列表更松。
class InfoRow extends StatelessWidget {
  const InfoRow(
    this.label,
    this.value, {
    super.key,
    this.valueColor,
    this.trailing,
  });

  final String label;
  final String value;
  final Color? valueColor;
  final Widget? trailing;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: AppSpace.gapS),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Expanded(
            flex: 2,
```

```
            child: Text(label,
                style: AppText.body.copyWith(color: AppColors.textSecondary)),
          ),
          const SizedBox(width: 12),
          Expanded(
            flex: 3,
            child: Text(
              value,
              textAlign: TextAlign.right,
              style: AppText.body.copyWith(
                fontWeight: FontWeight.w500,
                color: valueColor ?? AppColors.textPrimary,
              ),
            ),
          ),
          if (trailing != null) ...[
            const SizedBox(width: 6),
            trailing!,
          ],
        ],
      ),
    );
  }
}

/// 行内分隔线。比 Divider 更淡，用于卡片内的条目之间。
class RowDivider extends StatelessWidget {
  const RowDivider({super.key});

  @override
  Widget build(BuildContext context) =>
      const Divider(height: 1, thickness: 1, color: AppColors.divider);
}

/// 性别符号 ♀ / ♂。
///
/// 抽之前 `profile_screen` 与 `today_screen` 各有一份 `_GenderMark`，
/// **逐字相同**，只有 `fontSize` 差 1（17 vs 16）—— 这种「只差一个字面量」
/// 的拷贝是最难发现的：改配色时改一份，另一份就悄悄留旧色。
///
/// 未知性别返回空盒子。**不要凭空画一个符号出来 —— 那是编数据。**
///
/// 配色有意写在这里而不是主题里：粉色/蓝色是符号自身的通用语义
/// （♀ 粉、 ♂ 蓝），不是 App 的语义色，换成主题色反而认不出来。
class GenderMark extends StatelessWidget {
  const GenderMark({super.key, required this.gender, this.fontSize = 17});

  final String? gender;
  final double fontSize;
```

```
  @override
  Widget build(BuildContext context) {
    final g = (gender ?? '').trim().toLowerCase();
    final (symbol, color) = switch (g) {
      'female' || 'f' || '母' || '雌' => ('♀', const Color(0xFFEC4899)),
      'male' || 'm' || '公' || '雄' => ('♂', const Color(0xFF3B82F6)),
      _ => ('', AppColors.textTertiary),
    };
    if (symbol.isEmpty) return const SizedBox.shrink();
    return Text(
      symbol,
      style: TextStyle(fontSize: fontSize, height: 1, color: color),
    );
  }
}

/// 带图标的统计小块。参考稿的「体重 / 状态 / 活跃度」三格用的就是这个。
class StatTile extends StatelessWidget {
  const StatTile({
    super.key,
    required this.icon,
    required this.value,
    required this.label,
    this.unit,
    this.tint,
  });

  final IconData icon;
  final String value;
  final String label;
  final String? unit;
  final Color? tint;

  @override
  Widget build(BuildContext context) {
    return Row(
      crossAxisAlignment: CrossAxisAlignment.center,
      children: [
        Container(
          width: 28,
          height: 28,
          decoration: BoxDecoration(
              color: tint ?? AppColors.primaryLight,
              borderRadius: BorderRadius.circular(8)),
          child: Icon(icon, size: 16, color: AppColors.primary),
        ),
        const SizedBox(width: 8),
        Expanded(
            child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
```

```
          children: [
            Text.rich(
                TextSpan(text: value, children: [
                  if (unit != null)
                    TextSpan(text: ' $unit', style: AppText.caption),
                ]),
                style: const TextStyle(
                    fontSize: 17,
                    fontWeight: FontWeight.w600,
                    color: AppColors.textPrimary,
                    height: 1.25)),
            const SizedBox(height: 3),
            Text(label,
                maxLines: 2,
                overflow: TextOverflow.ellipsis,
                style: const TextStyle(
                    fontSize: 12, color: AppColors.textSecondary, height: 1.4)),
          ],
        )),
      ],
    );
  }
}

/// 小圆角标签。参考稿的「健康良好」「友善」「活泼」都是这种。
class SoftTag extends StatelessWidget {
  const SoftTag(
    this.text, {
    super.key,
    this.color,
    this.bg,
    this.icon,
  });

  final String text;
  final Color? color;
  final Color? bg;
  final IconData? icon;

  @override
  Widget build(BuildContext context) {
    final fg = color ?? AppColors.primary;
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 9, vertical: 4),
      decoration: BoxDecoration(
        color: bg ?? AppColors.primaryLight,
        borderRadius: BorderRadius.circular(AppRadius.chip),
      ),
      child: Row(
        mainAxisSize: MainAxisSize.min,
```

```
        children: [
          if (icon != null) ...[
            Icon(icon, size: 12, color: fg),
            const SizedBox(width: 4),
          ],
          Flexible(
              child: Text(
            text,
            style: TextStyle(
              fontSize: 11.5,
              fontWeight: FontWeight.w500,
              color: fg,
            ),
          )),
        ],
      ),
    );
  }
}

/// 头像。有图显示图，没图用种类 emoji 占位。
///
/// [borderWidth] > 0 时加白色描边 —— 参考稿在渐变背景上用了这个效果。
class PetAvatar extends StatelessWidget {
  const PetAvatar({
    super.key,
    required this.pet,
    this.size = 44,
    this.borderWidth = 0,
  });

  final Pet pet;
  final double size;
  final double borderWidth;

  @override
  Widget build(BuildContext context) {
    final emoji = switch (pet.species) {
      Species.dog => '',
      Species.cat => '',
      Species.other => '',
    };

    final fallback = Container(
      width: size,
      height: size,
      alignment: Alignment.center,
      decoration: const BoxDecoration(
        color: AppColors.primaryLight,
        shape: BoxShape.circle,
```

```
      ),
      child: Text(emoji, style: TextStyle(fontSize: size * 0.48)),
    );

    final inner = _localAvatar(size) ?? fallback;

    if (borderWidth <= 0) return inner;

    return Container(
      padding: EdgeInsets.all(borderWidth),
      decoration: const BoxDecoration(
        color: Colors.white,
        shape: BoxShape.circle,
      ),
      child: inner,
    );
  }

  /// 本地头像文件。三种情况都回落到 emoji：
  /// - 没设置
  /// - 值是远端 URL（同步功能写入的，本机还没下载）
  /// - 文件被系统/用户清了
  ///
  /// 用 `errorBuilder` 兜住「文件存在但解不开」（半个文件、格式坏了），
  /// 否则头像会变成一个红叉，比没头像更难看。
  Widget? _localAvatar(double size) {
    final path = (pet.avatarUrl ?? '').trim();
    if (path.isEmpty || path.startsWith('http')) return null;
    final file = File(path);
    if (!file.existsSync()) return null;

    return ClipOval(
      child: Image.file(
        file,
        width: size,
        height: size,
        fit: BoxFit.cover,
        errorBuilder: (_, __, ___) => Container(
          width: size,
          height: size,
          color: AppColors.primaryLight,
        ),
      ),
    );
  }
}

// ---------------------------------------------------------------- 文案工具

IconData recordTypeIcon(RecordType type) => switch (type) {
```

```
      RecordType.weight => Icons.monitor_weight_outlined,
      RecordType.vaccine => Icons.vaccines_outlined,
      RecordType.dewormInternal => Icons.medication_outlined,
      RecordType.dewormExternal => Icons.bug_report_outlined,
      RecordType.medication => Icons.medication_liquid_outlined,
      RecordType.medical => Icons.local_hospital_outlined,
      RecordType.symptom => Icons.health_and_safety_outlined,
      RecordType.grooming => Icons.shower_outlined,
      RecordType.feeding => Icons.restaurant_outlined,
      // 注意 toilet 已经占了 water_drop，饮水另用一个杯子图标。
      RecordType.water => Icons.local_drink_outlined,
      RecordType.toilet => Icons.water_drop_outlined,
      RecordType.sleep => Icons.bedtime_outlined,
      RecordType.note => Icons.sticky_note_2_outlined,
    };

/// 费用类别 → 图标。
///
/// 与 [recordTypeIcon] 隔开：费用是「钱花在哪」，记录是「发生了什么」。
/// 主粮用米袋而不是餐厅盘（那是喂食记录），用品用购物袋，寄养用房子。
IconData expenseCategoryIcon(ExpenseCategory c) => switch (c) {
      ExpenseCategory.food => Icons.rice_bowl_outlined,
      ExpenseCategory.medical => Icons.local_hospital_outlined,
      ExpenseCategory.vaccine => Icons.vaccines_outlined,
      ExpenseCategory.deworm => Icons.bug_report_outlined,
      ExpenseCategory.grooming => Icons.shower_outlined,
      ExpenseCategory.supply => Icons.shopping_bag_outlined,
      ExpenseCategory.boarding => Icons.night_shelter_outlined,
      ExpenseCategory.other => Icons.more_horiz_rounded,
    };

/// 秒数 → 「1 小时 23 分」/「1h 23m」。
///
/// 遛狗结果、档案页的遛狗列表都要这一段，各写一份迟早会长出两个格式。
String durationLabel(int seconds) {
  final (h, m) = Units.splitDuration(seconds);
  if (h > 0) return L.isZh ? '$h 小时 $m 分' : '${h}h ${m}m';
  return L.isZh ? '$m 分钟' : '${m}m';
}

/// 遛狗心情的五种取值。存 code，不存 emoji —— emoji 有可能变，code 不会。
const List<String> kWalkMoods = ['great', 'good', 'tired', 'anxious', 'sick'];

String walkMoodEmoji(String code) => switch (code) {
      'good' => '',
      'tired' => '',
      'anxious' => '',
      'sick' => '',
      _ => '',
    };
```

```
String walkMoodLabel(String code) => switch (code) {
      'good' => L.t('walk.mood.good'),
      'tired' => L.t('walk.mood.tired'),
      'anxious' => L.t('walk.mood.anxious'),
      'sick' => L.t('walk.mood.sick'),
      _ => L.t('walk.mood.great'),
    };

/// 提醒标题：库里存的是 i18n key，展示时才翻。
/// 提醒标题。解析规则见 core/reminder_text.dart —— 通知服务也要用同一套，
/// 所以那边才是实现，这里是给界面用的薄封装。
String reminderTitle(Reminder r) => reminderTitleFrom(r.title, r.type);

/// 相对时间：今天 / 昨天 / N 天前 / 具体日期。
String relativeDay(DateTime dt) {
  final now = DateTime.now();
  final today = DateTime(now.year, now.month, now.day);
  final that = DateTime(dt.year, dt.month, dt.day);
  final diff = today.difference(that).inDays;

  if (diff == 0) return L.t('timeline.today');
  if (diff == 1) return L.t('timeline.yesterday');
  if (diff > 1 && diff < 30) return L.tp('timeline.daysAgo', {'n': diff});
  return '${dt.year}-${pad2(dt.month)}-${pad2(dt.day)}';
}

/// 提醒倒计时：「还有 3 天」/「已过期 2 天」/「今天」
String dueLabel(DateTime dueAt) {
  final now = DateTime.now();
  final today = DateTime(now.year, now.month, now.day);
  final that = DateTime(dueAt.year, dueAt.month, dueAt.day);
  final diff = that.difference(today).inDays;

  if (diff == 0) return L.t('timeline.today');
  if (diff == 1) return L.t('due.tomorrow');
  if (diff > 1) return L.tp('due.inDays', {'n': diff});
  return L.tp('due.overdueBy', {'n': -diff});
}

/// 日期时间紧凑格式。
String compactDateTime(DateTime dt) =>
    '${pad2(dt.month)}-${pad2(dt.day)} ${pad2(dt.hour)}:${pad2(dt.minute)}';

/// 文档原件 → 图标，按扩展名分。
///
/// 只认扩展名不认 MIME：MIME 是我们自己从扩展名推出来的，看它等于绕一圈；
/// 而用户认的「PDF」就是文件后缀那三个字母。
IconData documentIcon(String? ext) =>
    switch ((ext ?? '').toLowerCase().replaceFirst('.', '')) {
```

```
      'pdf' => Icons.picture_as_pdf_outlined,
      'doc' || 'docx' || 'txt' || 'rtf' => Icons.description_outlined,
      'xls' || 'xlsx' || 'csv' => Icons.table_chart_outlined,
      'jpg' ||
      'jpeg' ||
      'png' ||
      'heic' ||
      'heif' ||
      'webp' ||
      'gif' =>
        Icons.image_outlined,
      _ => Icons.insert_drive_file_outlined,
    };

/// 日期紧凑格式（只有年月日）。
///
/// 与 [compactDateTime] 分开：费用、疫苗本这类「发生在哪天」的场合
/// 显示几点几分是噪声，而且会让人以为时间是有意义的。
String compactDate(DateTime dt) => L.isZh
    ? '${dt.year}年${dt.month}月${dt.day}日'
    : '${pad2(dt.month)}/${pad2(dt.day)}/${dt.year}';

/// 台账分类 → 图标。转发给 [reminderTypeIcon]，同一件事在两处不能长两个样。
IconData careKindIcon(PlanItemType kind) => reminderTypeIcon(kind.wireName);

/// 提醒类型 → 小图标。类型是 wire 字符串，不是 RecordType 枚举。
///
/// 提升为顶层函数是为了让 Upcoming 行也能复用同一套图标映射，
/// 免得今日卡和未来列表对同一个类型画出两个不同的图标。
/// 提醒类型 → 图标。
///
/// 同时认两种写法：**库里存的是下划线式**（`deworm_internal`，来自
/// PlanItemType.wireName），而早年这里只匹配驼峰式，导致驱虫和体检
/// 一直显示成默认的小铃铛。别把下划线那组删掉。
IconData reminderTypeIcon(String type) => switch (type) {
      'vaccine' => Icons.vaccines_outlined,
      'dewormInternal' || 'deworm_internal' => Icons.medication_outlined,
      'dewormExternal' || 'deworm_external' => Icons.bug_report_outlined,
      'checkup' => Icons.health_and_safety_outlined,
      'grooming' => Icons.shower_outlined,
      'medication' => Icons.medication_liquid_outlined,
      'medical' => Icons.local_hospital_outlined,
      'feeding' => Icons.restaurant_outlined,
      'toilet' => Icons.water_drop_outlined,
      _ => Icons.notifications_none_rounded,
    };

/// 待办小卡（今日页横向滚动用）。
///
/// 版式取自参考稿：图标底 + 标题 + 时间/倒计时两行。
```

```
/// 每张卡按类型配不同底色，扫一眼就能分出维生素、晚餐、驱虫。
///
/// 卡片只负责「看」，操作按钮在卡片外的操作条上（见 today_screen 的 _TodoActions）。
/// 原因：横向卡固定 132px 高，塞不下按钮；而「一键完成」不能丢。
class TodoTile extends StatelessWidget {
  const TodoTile({
    super.key,
    required this.reminder,
    this.petName,
    this.overdue = false,
  });

  final Reminder reminder;
  final String? petName;
  final bool overdue;

  @override
  Widget build(BuildContext context) {
    // 按类型散列到一组底色，保证同一类型每次拿到的颜色一致。
    final tint = overdue
        ? AppColors.dangerBg
        : AppColors.tileTints[
            reminder.type.hashCode.abs() % AppColors.tileTints.length];
    final fg = overdue ? AppColors.danger : AppColors.primary;

    return Container(
      width: 152,
      padding: const EdgeInsets.all(AppSpace.gapM),
      decoration: BoxDecoration(
        color: AppColors.surface,
        borderRadius: AppRadius.cardBorder,
        border: Border.all(color: AppColors.border),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Container(
            width: 32,
            height: 32,
            decoration: BoxDecoration(
              color: tint,
              borderRadius: BorderRadius.circular(10),
            ),
            child: Icon(reminderTypeIcon(reminder.type), size: 17, color: fg),
          ),
          const SizedBox(height: AppSpace.gapM),
          Text(
            reminderTitle(reminder),
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
```

```
            style: const TextStyle(
              fontSize: 13,
              fontWeight: FontWeight.w600,
              color: AppColors.textPrimary,
            ),
          ),
          const SizedBox(height: 3),
          Text(
            _when(reminder),
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
            style: TextStyle(
              fontSize: 11.5,
              color: overdue ? AppColors.danger : AppColors.textSecondary,
            ),
          ),
          if (petName != null) ...[
            const Spacer(),
            Text(
              petName!,
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: const TextStyle(
                fontSize: 10.5,
                color: AppColors.textTertiary,
              ),
            ),
          ],
        ],
      ),
    );
  }

  /// 参考稿这里显示「08:00」这样的具体时刻，过期项显示「已过期 N 天」。
  static String _when(Reminder r) {
    final now = DateTime.now();
    final sameDay = r.nextAt.year == now.year &&
        r.nextAt.month == now.month &&
        r.nextAt.day == now.day;

    if (sameDay) {
      return '${pad2(r.nextAt.hour)}:${pad2(r.nextAt.minute)}';
    }
    return dueLabel(r.nextAt);
  }
}

// ==================== android/app/src/main/kotlin/com/weiyuantool/pet_app/MainActivity.kt ==============
package com.weiyuantool.pet_app
```

```
import android.content.Intent
import android.net.Uri
import android.os.Build
import android.provider.Settings
import androidx.core.content.FileProvider
import io.flutter.embedding.android.FlutterActivity
import io.flutter.embedding.engine.FlutterEngine
import io.flutter.plugin.common.MethodChannel
import java.io.File

/**
 * 应用内更新的原生侧：把下载好的 APK 交给系统安装器。
 *
 * Dart 侧做不了这件事，只能走平台通道：
 * 1. 拉起安装界面需要 Intent，Dart 没有对应能力；
 * 2. Android 7.0 起必须把文件包成 content:// URI（FileProvider），
 *    直接用 file:// 会抛 FileUriExposedException。
 */
class MainActivity : FlutterActivity() {

    private companion object {
        const val CHANNEL = "com.weiyuantool.pet_app/installer"

        /** APK 的 MIME 类型。装包必须带它，否则系统不知道该由谁处理。 */
        const val APK_MIME = "application/vnd.android.package-archive"
    }

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)

        MethodChannel(flutterEngine.dartExecutor.binaryMessenger, CHANNEL)
            .setMethodCallHandler { call, result ->
                when (call.method) {
                    "installApk" -> {
                        val path = call.argument<String>("path")
                        result.success(path != null && installApk(path))
                    }

                    "openInstallSettings" -> {
                        openInstallSettings()
                        result.success(null)
                    }

                    else -> result.notImplemented()
                }
            }
    }

    /**
     * 返回 false 表示**没能拉起安装器**（缺授权或文件不在），
```

```
     * Dart 侧收到 false 会提示用户去系统设置授权。
     */
    private fun installApk(path: String): Boolean {
        return try {
            // Android 8.0+ 的「安装未知应用」是按应用单独授权的。
            // 没授权就 startActivity，系统会静默丢弃这个 Intent，
            // 用户只看到「点了没反应」—— 所以这里必须先查、先引导。
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O &&
                !packageManager.canRequestPackageInstalls()
            ) {
                openInstallSettings()
                return false
            }

            val file = File(path)
            if (!file.exists()) return false

            // authority 必须与 AndroidManifest 里声明的完全一致，
            // 拼错会抛 IllegalArgumentException，表现同样是「点了没反应」。
            val uri = FileProvider.getUriForFile(this, "$packageName.fileprovider", file)

            val intent = Intent(Intent.ACTION_VIEW).apply {
                setDataAndType(uri, APK_MIME)
                addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            }
            startActivity(intent)
            true
        } catch (e: Exception) {
            false
        }
    }

    /** 跳到本应用的「安装未知应用」授权页。 */
    private fun openInstallSettings() {
        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) return
        try {
            startActivity(
                Intent(Settings.ACTION_MANAGE_UNKNOWN_APP_SOURCES).apply {
                    data = Uri.parse("package:$packageName")
                    addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
                }
            )
        } catch (_: Exception) {
            // 个别 ROM 没有这个页面，忽略即可 —— 用户也能自己去设置里找。
        }
    }
}
```